计算机导论

Introduction to Computer

作者：是晓诺诺吖

rmolives@gmail.com

www.rmolives.com

时间：July 2, 2022

我们需要努力，并且不浪费努力

目录

第 1 章 程序 **2**
1.1 作用域与变量 3
1.2 条件表达式 4
1.3 递归 5
1.4 高阶函数 6
1.5 黑箱 6
1.6 解释原理 6

第 2 章 初等数论 **8**
2.1 整除性与最大公因数 8
2.2 素数 9
2.3 算术基本定理 9
2.4 欧拉函数 11
2.5 同余式 12
2.6 费马小定理 12
2.7 欧拉定理 12
2.8 贝祖定理 12
2.9 乘法逆元 13
2.10 线性同余方程 13
2.11 模 p 平方剩余 13
2.12 斐波纳契数列 16
2.13 素数判定 17
2.14 高精度计算 18

第 3 章 计算理论 **20**
3.1 自动机前置定义 20
3.2 确定性有穷自动机 22
3.3 非确定性有穷自动机 24
3.4 带空转移的非确定性有穷自动机 26
3.5 状态转移表 28
3.6 正则语言 28
3.7 NFA 与 DFA 识别能力等价 28
3.8 子集构造法 29
3.9 时间复杂度 30

3.10 P 与 NP 与 NPC 31

第 4 章 算法与数据结构 32

4.1 快速幂与快速乘 32
4.2 前缀和与差分 33
4.3 栈与队列 34
4.4 链表 35
4.5 二叉树 36
4.6 二叉搜索树 37
4.7 深度优先搜索 37
4.8 广度优先搜索 37
4.9 邻接表与邻接矩阵 38
4.10 单源最短路径 39
4.11 Dijkstra 算法 41
4.12 Bellman-Ford 算法 41
4.13 哈密顿回路 42
4.14 旅行商问题 42
4.15 矩阵 42
4.16 矩阵加速递推 42
4.17 哈希树 43
4.18 二进制状态压缩 43
4.19 动态规划 43
4.20 背包 45
4.21 并查集 45

第 5 章 密码学 46

5.1 随机数生成器 46
5.2 哈希函数 47
5.3 古典密码 47
5.4 序列密码 48
5.5 一次一密 48
5.6 对称加密 48
5.7 非对称加密 48
5.8 数字签名 49

第 6 章 计算机组成 50

6.1 冯 · 诺依曼计算机 50
6.2 存储器概念 50

6.3 寄存器 51
6.4 随机存储器 51
6.5 只读存储器 52
6.6 闪存 53
6.7 哈佛结构 54
6.8 高速缓冲存储器 54

第 7 章 组合数学 **55**
7.1 加法原理 55
7.2 乘法原理 55
7.3 排列数 55
7.4 组合数 55
7.5 二项式定理 56
7.6 卢卡斯定理 56
7.7 卡塔兰数列 56
7.8 容斥原理 56
7.9 莫比乌斯函数 56

第 8 章 杂项 **57**
8.1 程序正确性证明 57
8.2 区块链 59

Introduction

这是一本介绍计算机的书。基于经验向有一定计算机素养的人传授计算机知识。本书并不适合那些在计算机领域深耕多年的老手, 也不适合完全零基础的初学者, 仅适合想进一步学习计算机而无从下手的人。面对知识庞大且复杂的计算机, 你很可能会感到沮丧而放弃。本书将循序渐进的带你进入计算机的世界, 让你感受这一美丽的学科。

在本书中, 我们将会使用 **Hime 语言**和 **C 语言**作为讲解语言, **Hime 语言**是由**是晓诺诺吖**, 也就是本书作者创造的类 **Lisp** 语言。我们会在**第一章**使用 **Hime 语言**作为讲解语言来讲解一些基本的概念, 其后将使用 **C 语言**作为讲解语言。

阅读本书请确保你具备高中及以上的素质和知识素养, 也了解基本的计算机知识, 比如内存、CPU 是什么, 一些基本的程序开发能力之类的。这样更方便阅读和理解本书的内容, 更易确保你能够实际掌握重要概念。

我们需要努力, 并且不浪费努力。

是晓诺诺吖
rmolives@gmail.com
Guangxi
July 2, 2022

第 1 章 程序

内容提要

- 环境是一系列绑定 (binding)
- 对于一个编程语言，通常有解释器的实现或编译器的实现
- 无需关注这个函数是如何让计算出它的结果的，只需要关注它能够计算出正确的结果的事实

那么，什么是**程序**？为了在计算机上描述我们需要的计算过程，而设计的一类能被计算机理解或间接理解的语言，这种语言被称为**程序设计语言**，或者**编程语言**。正如人们每天使用**自然语言**进行对话，而我们与计算机进行对话使用的则是**编程语言**。而利用这些语言编写的东西就被称为**程序**。

我们使用的编程语言是 **Hime**，这是一门类 **Lisp** 语言。

你可以通过https://github.com/RMOlive/Hime下载并使用这一门编程语言。

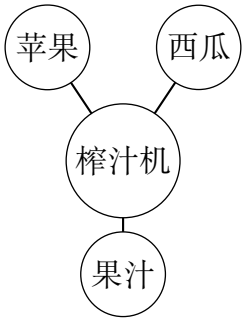
Hime 的基本语法规则是

```
(<function-name> <argument*>)
```

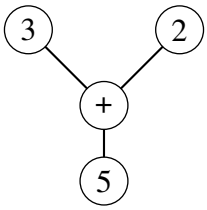
比如说对于一个程序：

```
(+ 3 2)
```

我们可以理解成是把 **+** 运用到 **3** 和 **2** 上，或者说是把 **3** 和 **2** 丢进 **+** 里面。计算机会对其进行求值，并得出结果。这就好像一台**榨汁机**，我们把**苹果**和**西瓜**丢进**榨汁机**里，得到一杯**果汁**。



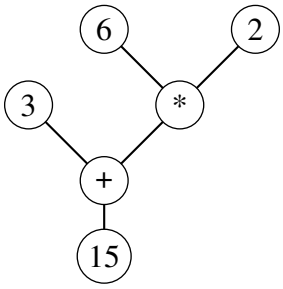
而对于如上程序 **(+ 3 2)** 也是一样，我们把 **3** 和 **2** 丢进 **+** 里面，得到最后的结果。



我们也可以进行嵌套计算，例如：

```
(+ 3 (* 6 2))
```

则运算过程是先把 **6** 和 **2** 丢进 ***** 里, 再把**结果**和 **3** 丢进 **+** 里。



对于编写程序, 我们要建立一种**环境 (environment)** 的概念, **环境**是一系列**绑定 (binding)**, 而**绑定**则是从**标识符 (identifier)** 到**内容 (value)** 之间的映射。

我们可以使用 **def** 来建立一个新的**绑定**, 然后通过它的**标识符**来使用。其规则是:

```
(def <identifier> <value>)
```

例如对于 $a = 3$, 则可以通过 **(def a 3)** 来建立绑定。而要计算 $a + 2$, 则可使用 **(+ a 2)** 来计算。

而对于函数 $f(x) = x * 2$ 来说, 则可以使用规则:

```
(def (<name> <parmeter*>) <body*>)
```

写成 **(def (f x) (* x 2))**

当然, 函数也可以使用 **lambda** 来定义, 其形式为:

```
(lambda (<parmeter*>) <body*>)
```

这样的函数没有名字, 只可以通过别名或者直接使用的方式来使用。例如:

```
((lambda (x) (+ x 3)) 2)
```

而上面的函数 **f** 则可使用 **lambda** 写作:

```
(def f (lambda (x) (* x 2)))
```

1.1 作用域与变量

至此, 我们可以引入一个新的概念, 叫做**作用域 (scope)**, 作用域分为两种, 第一种是**动态作用域 (Dynamic Scope)**, 此时调用的函数使用的环境, 取决于函数的调用位置, 第二中是**静态作用域 (Static Scope)**, 取决于函数的定义位置, 或者说取决于上下文。所以, **静态作用域 (Static Scope)** 又叫**词法作用域 (Lexical Scope)**。

而与此相关的概念还有**自由变量**与**约束变量**, 简单来说, 局部变量和参数都被认为是**约束变量**; 而不是**约束变量**的则是**自由变量**。比如说对于 $\sum_{x=1}^{10} f(x,y)$, x 就是**约束变量**, 而 y 则是**自由变量**。

比如对于程序:

```
(def v 2)
(def (f n) (+ v n))
(def (test)
    (def v 3)
    (f 2))
```

如果采用**动态作用域**, 则 **(test)** 的执行结果为 5, 而采用**静态作用域**的话, 执行结果则为 4。早期的 **Lisp** 采用**动态作用域**, 较新的**编程语言**如 **Scheme** 则多采用**静态作用域**, **Hime** 也是采用**静态作用域**的。请借助搜索引擎认真理解**静态作用域**与**动态作用域**的区别。

其中对于函数 f 来说, n 是**约束变量**, 而 v 则是**自由变量**。

1.2 条件表达式

对于计算, 我们经常出现对于某一种情况执行一段程序, 而对于另一种情况执行另一端程序的需求, 比如**绝对值**。此时我们就可以使用**条件表达式**来实现。

Hime 中有两种可以实现如上需求的形式, 一种是 **if**, 形式为:

```
(if <predicate> <consequent> <alternative>)
```

例如对于计算绝对值 $abs(n) = \begin{cases} n & n \geq 0 \\ -n & \end{cases}$ 使用 **Hime** 可写作:

```
(def (abs n)
    (if (>= n 0)
        n
        (- n)))
```

而另一种是 **cond**, 形式为:

```
(cond (<p1> <e1>)
      (<p2> <e2>)
      ...
      (<pn> <en>))
```

cond 用于多种条件的匹配执行。

而对于绝对值, 则可写成:

```
(def (abs n)
   (cond ((>= n 0) n)
         ((< n 0) (- n))))
```

此时假如调用 **(abs -3)**, 计算机会首先检查 **(>= -3 0)** 是否满足, 显然结果是不满足。于是就跳到下一条, 检查 **(< -3 0)**, 成立, 返回 **(- -3)**, 也就是 3。

其中有个特殊的符号 **else**, 可以放在末尾, 每次匹配完所有的条件, 结果都不满足, 就会执行 **else** 的内容。也就是上面的程序可写成:

```
(def (abs n)
   (cond ((>= n 0) n)
         (else (- n))))
```

我们可以使用 **and**、**or**、**not** 来条件进行**与**、**或**、**非**操作。

比如我们想判断**条件 A** 和**条件 B** 都成立, 则可写作 **(and A B)**, 而要判断**条件 C** 不成立的情况, 则可写作 **(not C)**。

1.3 递归

递归 (recursion) 是指函数的定义中使用自身的方法, 我们可以拿阶乘举例子, 阶乘在数学上递归定义为:

$$fact_n = \begin{cases} 1 & n = 1 \\ n * fact(n-1) & \end{cases}$$

使用 **Hime** 则可写成:

```
(def (fact n)
   (if (= n 1)
       1
       (* n (fact (- n 1)))))
```

这就是**递归**。

除此之外, 递归还额外指一种通过重复将问题分解为同类的子问题而解决问题的方法。

比如著名的**斐波纳契数列**, 我们在求第 n 项的时候, 就是反复将其划分为相同的子问题进行求解。其第 n 项递归定义为:

$$fib_n = \begin{cases} 1 & n < 2 \\ fib_{n-1} + fib_{n-2} & \end{cases}$$

使用 **Hime** 实现则为:

```
(def (fact n)
   (if (< n 2)
      n
      (+ (fib (- n 1)) (fib (- n 2)))))
```

1.4 高阶函数

高阶函数被定义为接受一个函数作为参数, 或者返回一个函数的函数。比如:

```
(def (apply f n) (f n))
```

就是接收 2 个参数, 一个函数和一个值, 并将这个函数运用在这个值上。

比如说 **map** 是传入的函数运用在列表的元素上, 然后获得返回值组成新的列表。例如:

```
(map (lambda (x) (* x 2)) (range 1 10))
```

返回的就是 **[2, 4, 6, 8, 10, 12, 14, 16, 18, 20]**。

1.5 黑箱

对于一个函数, 在理想情况下我们可以把它看作一个**黑箱**, 无需关注这个函数是如何让计算出它的结果的, 只需要关注它能够计算出正确的结果的事实。

1.6 解释原理

对于一个编程语言, 通常有**解释器**的实现或**编译器**的实现。对于**编译器**, 其目的就是把输入的代码翻译成能够被计算机执行的形式。而对于**解释器**, 其目的就是通过一个程序解析另一个程序代码进行执行。

简单说, **编译器**就是一个程序, 它可以阅读以某一种语言 **(源语言)** 编写的程序, 翻译成一个等价的、用另一种语言 **(目标语言)** 编写的程序。而**解释器**同样也是一个程序, 它不通过翻译的方式生成**目标语言**, 而是对**源语言**进行解释求值。

对于一个程序, 首先我们会对其进行**词法分析**, 其目的是把一整串程序变成一些被赋予类型的片段, 例如对于程序:

```
(+ 3 2)
```

我们可以将其划分成:

(	+	3	2	)

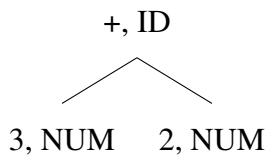
然后为其赋予类型：

(,LB	+, ID	3, NUM	2, NUM	),RB

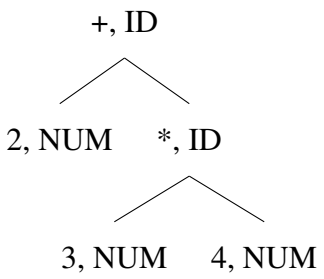
然后我们对其进行**语法分析**，生成出**抽象语法树**。

那么**抽象语法树**是什么？**抽象语法树**是源代码语法结构的一种抽象表示。它以树状的形式表现编程语言的语法结构，树上的每个节点都表示源代码中的一种结构。

上方程序**语法分析**生成的**抽象语法树**是：



而对于稍微复杂一点的嵌套，例如 **(+ 2 (* 3 4))**，则是：



而对其进行求值，实际上是一个树的遍历过程，我们先将 **3, NUM** 和 **4, NUM** 丢进 ***, ID** 里，得到结果 **12, NUM**，然后再将 **12, NUM** 和 **2,NUM** 丢进 **+, ID** 中，得到程序的最终结果 **14, NUM**。

而这一求值过程则是由**求值器**完成的。

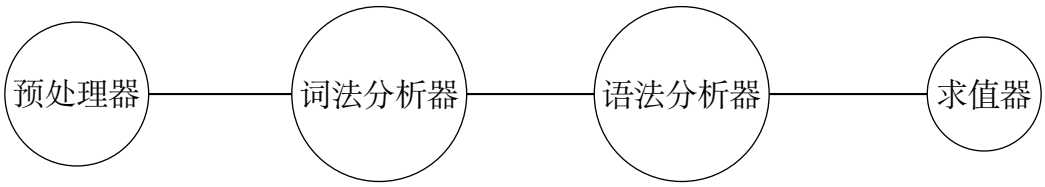
预处理器是负责产生能用来输入到词法分析器的数据的程序。比如在 **Hime** 中，**预处理器**是负责整理代码，把 **"不规范"** 的代码整理成规范代码。比如：

```
(def (fib n)
    (if (< n 2)
        n
        (+ (fib (- n 1)) (fib (- n 2)))))
```

则会被整理成：

```
(def (fib n) (if (< n 2) n (+ (fib (- n 1)) (fib (- n 2)))))
```

简而言之，解释器总体的处理流程如下：



第 2 章 初等数论

内容提要

- $a^p \equiv a \pmod p$
- $a^{\varphi}(n) \equiv 1 \pmod n$
- $gcd(m,n) * lcm(m,n) = m * n$
- $gcd(2m,2n) = 2gcd(m,n)$

我们常说的**数论**通常指代**初等数论**, 这是一门历史悠久的学科。早在**公元前 500 年**左右, 就有**毕达哥拉斯**将自然数区分为奇数、偶数、素数、完全数、平方数、三角数和五角数等。

本章将以**正整数**为研究对象, 带读者进入美妙而纯粹的数论世界。

2.1 整除性与最大公因数

假设 m 和 n 是整数, 且 $m \neq 0$, 则 m 整除 n 是指 n 是 m 的倍数。即存在整数 k 使得 $n = mk$, 如果 m 整除 n, 我们记为: $m \mid n$。类似的, 如果 m 不整除 n, 我们记为: $m \nmid n$。

为了方便如下讲解, 我们定义一个运算为 mod, 也就是**模除**。对于 $m \bmod n$, 我们将其定义为 $m \bmod n = m - n * \lfloor \frac{m}{n} \rfloor$, 其中 $\lfloor \frac{m}{n} \rfloor$ 为**向下取整**。

已知两个整数 m 和 n, 我们可以求其**公因数**, 即整除它们两个的数。例如, 由于 $4 \mid 12$ 且 $4 \mid 20$, 所以 4 是 12 与 20 的**公因数**。类似的, 3 是 18 和 30 的**公因数**。而我们把两个数最大的**公因数**称为**最大公因数**, 记为 $gcd(m,n)$。如果 $gcd(m,n) = 1$, 则称 m 与 n 互素。

2.1.1 欧几里得算法

求两个数**最大公因数**最有效的方法是**欧几里得算法**, 又叫**辗转相除法**。定义为:

$$gcd(m,n) = \begin{cases} m & n = 0 \\ gcd(n, m \bmod n) & \end{cases}$$

```
int gcd(m,n) {
    if (n == 0)
        return m;
    return gcd(n, m%n);
}
```

2.1.2 更相减损法

1. 对于所有的整数 m 与 n, $m > n$, 有 $gcd(m,n) = gcd(n, m-n) = gcd(m, m-n)$
2. 对于所有的整数 m 与 n, 有 $gcd(2m,2n) = 2gcd(m,n)$

2.1.3 性质

我们定义整数 m 与 n 的**最小公倍数**为 $lcm(m,n)$, 则有:

$$gcd(m,n) * lcm(m,n) = m * n$$

2.2 素数

素数又叫**质数**, 是指除了 1 和该数自身外无法被其他数整除的数, 反之我们称其为**合数**。常见的比如 2、3、5、7、11 等都是**素数**。

素数有无穷多个, 早在**公元前 300 多年**, 就有古希腊数学家**欧几里得**证明有无限多个**素数**存在。

素数判定是数论中十分基本的问题, 判定一个整数是否是**素数**, 最为直接朴素的想法就是利用素数的定义, 用小的**素数**去一一试除, 如果能整除的话, 就能确定为**合数**而非**素数**了。相比下节整数的**质因数分解**, **素数**判定一直以来被认为是较为容易的问题。

早期就 1 是否是**素数**展开的激烈的争辩, 比如**哥德巴赫**与**欧拉**著名的通信里将 1 列为第一个**素数**, 但**欧拉**不同意。到了 20 世纪初, 数学家开始认为 1 不是个**素数**, 但反而作为 "单位" 此一特殊类别。

2.2.1 试除法

如果 n 为**合数**, 则存在一个能整除 n 的数 t , 其中 $2 \le t \le \sqrt{n}$ 。

所以我们只需扫描 $2 \sim \sqrt{n}$ 之间的所有整数, 依次检查它们能否整除 n, 如果都不能整除, 则是**素数**, 否则是**合数**。需要特判 0 和 1 , 因为它们即不是**素数**, 也不是**合数**。

```
int is_prime(int n) {
    if (n < 2)
        return 0;
    for (int i = 2; i * i <= n; ++i)
        if (n % i == 0)
            return 0;
    return 1;
}
```

2.3 算术基本定理

大于 1 的正整数均可写成一个以上的**素数**之乘积, 且除了质因数的排序不同外是唯一的。

也就是任何一个大于 1 的正整数都能唯一分解为有限个**素数**的乘积, 可写作:

$$n = p_1^{c_1} p_2^{c_2} \cdots p_m^{c_m}$$

且满足 c_i 都是正整数, p_i 都是**素数**, 且满足 $p_1 < p_2 < \cdots < p_m$

其中 c_i 都是正整数, p_i 都是**素数**。我们通常将求解**算术基本定理**的行为称为**分解质因数**。

2.3.1 试除法

我们可以扫描 $2 \sim \lfloor \sqrt{n} \rfloor$ 的每个数 d, 如果 d 能整除 n, 则从 n 中除掉所有的因子 d, 同时累计除去的 d 的个数。

因为一个**合数**的因子一定在扫描到这个**合数**之前就从 n 中被除掉了, 所以在上述过程中能除 n 的一定是**素数**。

特别的, 如果 n 没被任何 $2 \sim \lfloor \sqrt{n} \rfloor$ 的数整除, 则 n 是**素数**, 无需分解。

```
void divide(int n) {
    for (int i = 2; i * i <= n; ++i)
        if (n % i == 0) {
            p[++m] = i, c[m] = 0;
            while (n % i == 0)
                n /= i, ++c[m];
        }
    if (n > 1)
        p[++m] = n, c[m] = 1;
}
```

2.3.2 算术基本定理推论

在**算术基本定理**中, 如果正整数 n 被唯一分解为 $n = p_1^{c_1} p_2^{c_2} \cdots p_m^{c_m}$, 其中 c_i 都是正整数, p_i 都是**素数**, 且满足 $p_1 < p_2 < \cdots < p_m$, 则 n 的正约数的集合可写作:

$$\{p_1^{b_1} p_2^{b_2} \cdots p_m^{b_m}\}$$

其中 $0 \le b_i \le c_i$

n 的正约数个数为:

$$\prod_{k=1}^{m} (c_k + 1)$$

n 的正约数之和为:

$$\prod_{k=1}^{m}\sum_{x=0}^{c_k}(p_k)^x$$

2.3.2.1 试除法

若 $d \geq \sqrt{n}$ 是 n 的约数, 则 $\frac{n}{d} \leq \sqrt{n}$ 也是 n 的约数。
因此, 只需扫描 $d = 1 \sim \sqrt{n}$, 尝试 d 能否整除 n, 如果能整除, 则 $\frac{n}{d}$ 也是 n 的约数。

```
void divisor(n) {
    for (int i = 1; i * i <= n; ++i)
        if (n % i == 0) {
            factor[++m] = i;
            if (i != n / i)
                factor[++m] = n / i;
        }
}
```

2.4 欧拉函数

$1 \sim n$ 中与 n 互素的数的个数被称为**欧拉函数**, 记为 $\varphi(n)$
若在**算数基本定理**中, $n = p_1^{c_1} p_2^{c_2} \cdots p_m^{c_m}$, 则:

$$\varphi(n) = n * \frac{p_1 - 1}{p_1} * \frac{p_2 - 1}{p_2} * \cdots * \frac{p_m - 1}{p_m} = n * \prod_{k=1}^{m} \frac{p_k - 1}{p_k}$$

根据**欧拉函数**的计算式, 我们只需要**分解质因数**, 即可求出**欧拉函数**。

```
int phi(int n) {
    int ans = n;
    for (int i = 2; i * i <= n; ++i)
        if (n % i == 0) {
            ans = ans / i * (i - 1);
            while (n % i == 0)
                n /= i;
        }
    if (n > 1)
        ans = ans / n * (n - 1);
    return ans;
}
```

2.4.1 性质

1. 对于 $n > 1, 1 \sim n$ 中与 n 互素的数的和为 $\frac{n*\varphi(n)}{2}$
2. 如果 m 与 n 互素, 则 $\varphi(mn) = \varphi(m)\varphi(n)$

2.5 同余式

我们通常把 $a \bmod c = b \bmod c$ 记为 $a \equiv b \pmod c$

2.6 费马小定理

如果 p 是素数, 则对于任意整数 a, 有 $a^p \equiv a \pmod p$

2.7 欧拉定理

如果正整数 a 与 n 互素, 则 $a^\varphi(n) \equiv 1 \pmod n$

2.7.1 欧拉定理的推论

如果正整数 a 与 n 互素, 则对于任意正整数 b, 有 $a^b \equiv a^{b \bmod \varphi(n)} \equiv 1 \pmod n$

2.8 贝祖定理

对于任意整数 a 与 b, 存在一对整数 x 与 y, 满足 $ax + by = gcd(a, b)$
x 与 y 的计算方法被称为**扩展欧几里得算法**。代码如下:

```
int exgcd(int a, int b, int &x, int &y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    int d = exgcd(b, a%b, x, y);
    int z = x;
    x = y;
    y = z - y * (a / b);
    return d;
}
```

2.9 乘法逆元

如果整数 b 与 m 互素, 并且 $b \mid a$, 则存在一个整数 x, 使得 $\frac{a}{b} \equiv a * x \pmod m$。称 x 为 b 的**模 m 乘法逆元**, 记为 $b^{-1} \pmod m$。

因为 $\frac{a}{b} \equiv a * b^{-1} \equiv \frac{a}{b} * b * b^{-1} \pmod m$, 所以 $b * b^{-1} \equiv 1 \pmod m$。

如果 m 是素数, 并且 $b < m$, 根据**费马小定理**, $b^{m-1} \equiv 1 \pmod m$, 即 $b * b^{m-2} \equiv 1 \pmod m$。

因此, 当模数 m 为素数时, b^{p-2} 即为 b 的**乘法逆元**。

如果只是保证 b 与 m 互素, 则可以通过求解线性同余方程 $b * x \equiv 1 \pmod m$ 得到。

2.10 线性同余方程

给定整数 a, b, m, 求一个整数 x, 满足 $a * x \equiv b \pmod m$, 或者给出无解。

$a * x \equiv b \pmod m$ 等价于 $a * x - b$ 是 m 的倍数, 不妨设为 $-y$ 倍。于是该方程可以改写为 $a * x + m * y = b$。

在有解时, 先用**欧几里德算法**求出一组整数 x_0, y_0, 满足 $a * x_0 + m * y_0 = gcd(a, m)$。然后

$$x = \frac{x_0 * b}{gcd(a, m)}$$

就是原**线性同余方程**的一个解。特别的, **线性同余方程**有解当且仅当 $gcd(a, m) \mid b$。

方程的通解则是所有模 $\frac{m}{gcd(a,m)}$ 与 x 同余的整数。

2.11 模 p 平方剩余

在上节, 我们已经学会如何解**线性同余方程**

$$a * x \equiv b \pmod m$$

现在决定转向**二次同余方程**。

首先我们要提出一个问题, 假设我们要求 $x^2 \equiv 4 \pmod 7$ 的解。

先使用一个简单粗暴的办法, 把 $1 \sim 7$ 中所有的数字代入 $a * x \equiv n \pmod 7$, 可以得到:

$$\begin{aligned}
1^2 &\equiv 1 \pmod 7 \\
2^2 &\equiv 4 \pmod 7 \\
3^2 &\equiv 2 \pmod 7 \\
4^2 &\equiv 2 \pmod 7 \\
5^2 &\equiv 4 \pmod 7 \\
6^2 &\equiv 1 \pmod 7
\end{aligned}$$

可见, $x^2 \equiv 4 \pmod 7$ 有 2 个解, 分别是 $x = 2$ 和 $x = 5$。

可是... 我们从这个表中只能得到这点信息吗? 好像... 是对称的? 没错, 它就是对称的。

如果 p 是素数, 当 $x^2 \equiv n \pmod p$ 存在一个解 n, 则另一个解为 $p - n$。

$$(p-n)^2 \equiv p^2 - 2 * p * n + n^2 \equiv n^2 \pmod p$$

2.11.1 二次剩余

若对于素数 p 和满足 $n \nmid p$ 的整数 n, 该方程的 x 有整数解, 则我们称 n 是关于模 p 的**二次剩余**, 记做 QR。而对于无整数解的称呼其为模 p 的**二次非剩余**。记做 NR。

而当 $n \equiv 0 \pmod p$ 的情况, 也就是 $n \mid p$, 这种情况下 n 既不是 QR 也不是 NR。

2.11.2 乘法法则

对于 QR 和 NR , 我们有如下性质。

1. $QR \times QR = QR$
2. $QR \times NR = NR$
3. $NR \times NR = QR$
4. $NR \times QR = NR$

2.11.3 勒让德符号

因为**二次剩余**的**乘法法则**性质与 1 和 -1 之间的性质很像, 所以我们可以设 $QR = 1$, $NR = -1$。

定义 n 模 p 的**勒让德符号**为:

$$\left(\frac{n}{p}\right) = \begin{cases} 1 & \text{if n is a QR modulo p and } n \not\equiv 0 \pmod p \\ -1 & \text{if n is a NR modulo p} \end{cases}$$

于是就得到了**乘法法则**的第二种表达形式:

$$\left(\frac{a}{p}\right)\left(\frac{b}{p}\right) = \left(\frac{a*b}{p}\right)$$

于是我们就得到了一个可以快速计算**勒让德符号**的方法, 或者说是快速判断是否是**二次剩余**的方法。例如:

$$\left(\frac{75}{97}\right) = \left(\frac{3*5*5}{97}\right) = \left(\frac{3}{97}\right)$$

我们可以注意到 $10^2 \equiv 3 \pmod{97}$

所以 $\left(\frac{75}{97}\right) = \left(\frac{3}{97}\right) = 1$

那么, 我们有没有什么办法不靠幸运或反复试验就能计算出像 $\left(\frac{3}{97}\right)$ 这样的**勒让德符号**呢? 答案是肯定的。

2.11.4 欧拉准则

设 p 是奇素数, 则:

$$a^{\frac{p-1}{2}} \equiv \left(\frac{a}{p}\right) \pmod{p}$$

2.11.5 二次互反律

设 p 和 q 是不同的奇素数, 则:

$$\left(\frac{-1}{p}\right) = \begin{cases} 1 & p \equiv 1 \pmod 4 \\ -1 & p \equiv 3 \pmod 4 \end{cases}$$

$$\left(\frac{2}{p}\right) = \begin{cases} 1 & (p \equiv 1 \pmod 8) \vee (p \equiv 7 \pmod 8) \\ -1 & (p \equiv 3 \pmod 8) \vee (p \equiv 5 \pmod 8) \end{cases}$$

$$\left(\frac{q}{p}\right) = \begin{cases} -\left(\frac{p}{q}\right) & (p \equiv 1 \pmod 4) \vee (q \equiv 1 \pmod 4) \\ \left(\frac{p}{q}\right) & (p \equiv 3 \pmod 4) \vee (q \equiv 3 \pmod 4) \end{cases}$$

于是我们就可以通过翻转**勒让德符号** $\left(\frac{q}{p}\right)$, 用 $\pm\left(\frac{p}{q}\right)$ 代替它, 然后模 q 简化 p, 不断的重复此过程, 这就导致求**勒让德符号**的数越来越小, 最终得到一个能够计算的**勒让德符号**。例如:

$$\begin{aligned}
\left(\frac{55}{179}\right) &= \left(\frac{5}{179}\right)\left(\frac{11}{179}\right) \\
&= \left(\frac{179}{5}\right) \times (-1) \times \left(\frac{179}{11}\right) \\
&= \left(\frac{4}{5}\right) \times (-1) \times \left(\frac{3}{11}\right) \\
&= 1 \times (-1) \times \left(\frac{3}{11}\right) \\
&= 1 \times (-1) \times (-1) \times \left(\frac{2}{3}\right) \\
&= 1 \times (-1) \times (-1) \times (-1) \\
&= -1
\end{aligned}$$

可能有些人看见上面的过程会有些疑惑, 比如 $\left(\frac{179}{5}\right) = \left(\frac{4}{5}\right)$, 这是因为 $179 \equiv 4 \pmod 5$ 比如:$\left(\frac{4}{5}\right) = 1$, 这是因为 $4 = 2^2$

2.11.6 Cipolla 算法

我们可以找到一个整数 a, 使得 $\left(\frac{a^2-n}{p}\right) = -1$, 定义 $w^2 = a^2 - n$

然后 $(a+w)^{\frac{p+1}{2}}$ 就是原方程的一个解, 而因为另一个解是在模 p 意义下对称的, 所以就为 $p-(a+w)^{\frac{p+1}{2}}$

2.12 斐波纳契数列

斐波那契数列又叫黄金分割数列, 在数学上是以递归进行定义的:

$$fib_n = \begin{cases} n & n < 2 \\ fib_{n-1} + fib_{n-2} & \end{cases}$$

2.12.1 通项公式

设常数 r 和 s 满足 $F_n - rF_{n-1} = s(F_{n-1} - rF_{m-2})$ 即 $F_n = (s+r)F_{n-1} - srF_{n-2}$

则 r 和 s 满足如下条件:

$$s + r = 1$$
$$s * r = -1$$

由韦达定理知, r 和 s 为一元二次方程 $x^2 - x - 1 = 0$ 的两个根, 不妨令

$$s = \frac{1+\sqrt{5}}{2}$$
$$r = \frac{1-\sqrt{5}}{2}$$

当 $n \geq 3$ 时, 有

$$\frac{F_n - rF_{n-1}}{F_{n-1} - rF_{n-2}} = s$$

则

$$\frac{F_3 - rF_2}{F_2 - rF_1} = s$$
$$\frac{F_4 - rF_3}{F_3 - rF_2} = s$$
$$\vdots$$
$$\frac{F_n - rF_{n-1}}{F_{n-1} - rF_{n-2}} = s$$

累乘得 $\frac{F_n - rF_{n-1}}{F_2 - rF_1} = s^{n-2}$

将 $F_{n-1}F_{n-2}$ 直到 F_3 按照上述递推关系式展开有:

$$\begin{aligned}
F_n &= s^{n-1} + rF_{n-1} = s^{n-1} + r(s^{n-2} + rF_{n-2}) \\
&= s^{n-1} + rs_{n-2} + r^2F_{n-2} = s^{n-1} + rs^{n-2} + r^2(s^{n-3} + rF_{n-3}) \\
&= \cdots \\
&= s^{n-1} + rs^{n-2} + r^2s^{n-3} + \cdots + r^{n-2}s + r^{n-1}
\end{aligned}$$

可见 F_n 是首项为 s^{n-1} , 公比为 $\frac{r}{s}$, 末项为 r^{n-1} 的等比数列求和, 根据等比数列求和公式有:

$$F_n = \frac{s^{n-1}(1 - \left(\frac{r}{s}\right)^n)}{1 - \frac{r}{s}} = \frac{s^n - r^n}{s - r}$$

将 r 和 s 代入:

$$\begin{aligned}
F_n &= \frac{\left(\frac{1+\sqrt{5}}{2}\right)^n - \left(\frac{1-\sqrt{5}}{2}\right)^n}{\frac{1+\sqrt{5}}{2} - \frac{1-\sqrt{5}}{2}} \\
&= \frac{1}{\sqrt{5}}\left(\left(\frac{1+\sqrt{5}}{2}\right)^n - \left(\frac{1-\sqrt{5}}{2}\right)^n\right)
\end{aligned}$$

即

$$F_n = \frac{1}{\sqrt{5}}\left(\left(\frac{1+\sqrt{5}}{2}\right)^n - \left(\frac{1-\sqrt{5}}{2}\right)^n\right)$$

2.13 素数判定

有些素数判定方法严格来说应该是**合性检测 (compositeness test)**, 这类方法总可以有效地把素数判定出来, 但可能把一个合数判定为素数。也就是说, 此类方法判定一个数为合数总是准确无误的, 而漏网的, 即无法判定的合数往往被称为**伪素数 (pseudoprimes)**。

在上文我们讲了**试除法**, 现在讲另外两种方法。

2.13.1 Fermat 检测

2.13.1.1 Fermat 小定理

若 p 为素数, $a \nmid p$, 则 $a^{p-1} \equiv 1 \pmod p$

2.13.1.2 Fermat 合性检验

任取整数 a, 若 $a \nmid N$ 且 $a^{N-1} \not\equiv 1 \pmod N$, 则输出 N 为合数, 否则输出 N 可能为素数。

2.13.1.3 Fermat 伪素数

满足 $a^{N-1} \not\equiv 1 \pmod N$ 的合数 N 称为一个对于基 a 的 **Fermat 伪素数**。

2.13.2 Euler 检测

2.13.2.1 Euler 合性检验

设 N 为奇素数, 任取整数 a, 满足 $a \nmid N$

1. 如果 $a^{\frac{N-1}{2}} \not\equiv \pm 1 \pmod p$, 则输出 N 为合数
2. $a^{\frac{N-1}{2}} \equiv \pm 1 \pmod p$ 且 $a^{\frac{N-1}{2}} \not\equiv \left(\frac{a}{N}\right) \pmod N$, 则输出 N 为合数
3. 如果 $a^{\frac{N-1}{2}} \equiv \left(\frac{a}{N}\right) \pmod N$, 则输出 N 可能为素数

2.13.2.2 Euler 伪素数

如果一个合数 N, $a \nmid N$, 满足 $a^{\frac{N-1}{2}} \equiv \left(\frac{a}{N}\right) \pmod N$, 则称 N 为对基 a 的 **Euler 伪素数**。

2.14 高精度计算

高精度运算 (Arbitrary-Precision Arithmetic) 是运用了一些算法结构来支持超过语言基本类型允许的更大整数间的运算。

2.14.1 存储

在平常的实现中, 高精度数字利用字符串表示, 每一个字符表示数字的一个十进制位。因此可以说, 高精度数值计算实际上是一种特别的字符串处理。正常读入字符串, 数字的最高位存储在开头, 但是为了方便近位等操作, 我们将反转字符串来存储, 同时将每一位字符转为数字。

例如对于字符串 **"12345"**, 我们通常存储为 **[5, 4, 3, 2, 1]**。

2.14.2 加减法

在我们小学学过**竖式加减法**, 在这里我们可以通过模拟**竖式加减法**来实现。

加法也就是从最低位开始, 将两个加数对应位置上的数码相加, 并判断是否达到或超过 10。如果达到, 那么处理进位: 将更高一位的结果上增加 1, 当前位的结果减少 10。

而减法则是从个位起逐位相减, 遇到负的情况则向上一位借 1。整体思路与加法完全一致。

2.14.3 乘除法

同**竖式加减法**, 我们也可以使用**竖式乘法**来实现。

回想竖式乘法的每一步, 实际上是计算了若干 $a \times b_i \times 10^i$ 的和。比如 1337×42 就是 $1337 \times 2^0 + 1337 \times 4 \times 10^4$。

而除法则也是**竖式除法**, **竖式除法**可以看作一个逐次减法的过程。

$$
\begin{array}{r}
38 \\
12\,\overline{)\,456} \\
\underline{36} \\
96 \\
\underline{96} \\
0
\end{array}
$$

将 45 减去 3 次 12, 不能再减, 于是此位是 3。$45 - 3 = 9$, 旁边的 6 移下来, 96 可以减去 8 次 12, 于是此位是 8, 没有剩余。

第 3 章 计算理论

内容提要

- O 定义的是一段程序执行步数的上界
- P 类问题是可以在多项式时间内解决的问题
- NP 类问题则是可以在多项式时间内验证的问题

计算理论大致可分为**计算可行性理论**和**计算复杂性理论**和**自动机理论**, 我们将就基本的**有穷状态自动机**入手, 延伸到**计算复杂性**, 浅浅的谈一谈著名的 **P=NP** 问题。

3.1 自动机前置定义

3.1.1 字母表

字母表是字母 (符号, 字符) 的非空有穷集。
比如字母表 $\Sigma_1 = \{0, 1\}$ 就是由符号 0 和 1 构成的字母表。

3.1.2 字符串

字符串就是由某个字母表中的符号组成的有穷序列。
比如上面的 $\Sigma_1 = \{0, 1\}$ 来说, $1, 10, 110, 10101$ 就是字母表 Σ_1 上的字符串。
Tips: 每个字符串都是有限长的。

3.1.3 空串

一般将空串记为 ε, 表示有 0 个字符的串。
字母表 Σ 可以是任意的, 都有 $\varepsilon \notin \Sigma$。

3.1.4 字符串的长度

我们将字符串 w 的长度记为 $|w|$, 定义如下:

$$|w| = \begin{cases} 0 & w = \varepsilon \\ |x| + 1 & w = xa \end{cases}$$

其中 $a \in \Sigma$,w 和 x 是字母表 Σ 中符号组成的字符串。
比如字母表 $\Sigma_1 = \{0, 1\}$ 上的字符串。

$$|101| = 3$$
$$|10111| = 5$$

3.1.5 字符串的连接

字符串 x 和 y 的连接：将首尾相接得到新字符串的运算, 记为 $x \cdot y$ 或 xy。

$$x \cdot y = \begin{cases} x & y = \varepsilon \\ (x \cdot z)a & y = za \end{cases}$$

其中 $a \in \Sigma$, x, y, z 都是字符串。
比如字母表 $\Sigma_1 = \{0, 1\}$ 上的字符串:

$$01 \cdot 10 = 0110$$
$$11 \cdot 00 = 1100$$

连接运算满足结合率, 即:

$$x \cdot y \cdot z = x \cdot (y \cdot z) = (x \cdot y) \cdot z$$

3.1.6 字符串的幂

符串的 n 次幂, 定义如下

$$x^n = \begin{cases} \varepsilon & n = 0 \\ x^{n-1} \cdot x & n > 0 \end{cases}$$

通俗讲, x^n 就是把 x 重复 n 次。
比如字母表 $\Sigma_1 = \{0, 1\}$ 上的字符串:

$$(1)^3 = 111$$
$$(01)^2 = 0101$$

3.1.7 集合的连接

集合 A 和 B 的连接, 记为 $A \cdot B$ 或 AB, 定义为

$$A \cdot B = w | w = x \cdot y, x \in A \wedge y \in B$$

例如

$$A = \{0, 11\}$$
$$B = \{ab, ba\}$$
$$A \cdot B = \{0ab, 0ba, 11ab, 11ba\}$$

3.1.8 集合的幂

集合 A 的 n 次幂 $(n \geq 0)$, 定义如下:

$$A^n = \begin{cases} \{\varepsilon\} & n = 0 \\ A^{n-1} \cdot A & n > 0 \end{cases}$$

打个比方, 对于字母表 $\Sigma_1 = \{0, 1\}$（字母表也是集合）

$$\Sigma_1^0 = \{\varepsilon\}$$
$$\Sigma_1^1 = \{0, 1\}$$
$$\Sigma_1^2 = \{00, 01, 10, 11\}$$

3.1.9 克林闭包

$$\Sigma^* = \bigcup_{i=0}^{\infty} \Sigma^i$$

3.1.10 正闭包

$$\Sigma^+ = \bigcup_{i=1}^{\infty} \Sigma^i$$

易知

$$\Sigma^* = \Sigma^+ \cup \varepsilon$$

3.2 确定性有穷自动机

确定性有穷自动机是一个 5 元组 $(Q, \Sigma, \delta, q_0, F)$, 其中:

1. Q 是一个有穷集合, 称为状态集
2. Σ 是一个有穷集合, 称为字母表
3. $\delta : Q \times \Sigma \to Q$ 是状态转移函数

4. $q_0 \in Q$ 是起始状态
5. $F \subseteq Q$ 是接受状态集

一台确定性有穷自动机有如上五个部分组成, 介绍如下:

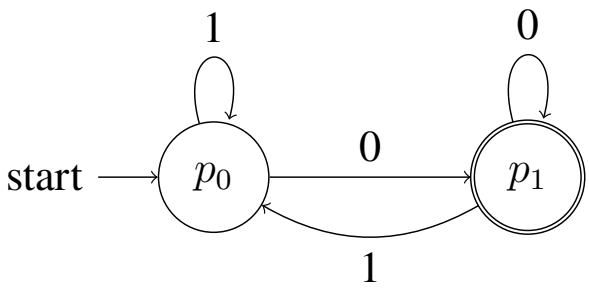
1. 它有一个状态集, 表示它有的全部状态
2. 它有一个输入字母表, 指明所有允许的输入符号
3. 它有一个根据一个输入字符从一个状态到另一个状态的规则
4. 它有一个起始状态, 表示处理所起始的状态
5. 它有一个接受状态集, 表示处理表达为接受的状态

3.2.1 图形表示

假如现在有个确定性有穷自动机 $\mathcal{M}_1 = (Q, \Sigma, \delta, q_0, F)$

$$\begin{cases} Q = \{q_0, q_1\} \\ \Sigma = \{0, 1\} \\ \delta = \begin{cases} \delta(q_0, 0) = q_1 \\ \delta(q_0, 1) = q_0 \\ \delta(q_1, 0) = q_1 \\ \delta(q_1, 1) = q_0 \end{cases} \\ F = \{q_1\} \end{cases}$$

图形化表示就是:



3.2.2 用处

主要对于输入的字符串判定是否该**确定性有穷自动机**所识别的语言, 对此表示接受及不接受。

而如上的**确定性有穷自动机**是识别以 0 结尾的串, 对于以 0 结尾的串 $x \in \Sigma^*$, 表示接受, 反之不接受。

确定性有穷自动机可以用来定义语言也可用于识别语言。

3.2.3 运算

假如对于上面的确定性有穷自动机 $\mathcal{M}_1$, 输入字符串 010, 此时从初始状态 q_0 开始, 读入第一个字符 0, 因为 $\delta(q_0, 0) = q_1$, 所以转移状态至 q_1, 继续读入下一个字符 1, 因为 $\delta(q_1, 1) = q_0$, 所以转移状态至 q_0, 继续读入下一个字符 0, 因为 $\delta(q_0, 0) = 1$ 所以转移状态至 q_1, 运算结束。因为最后停在状态 q_1, 因为 $q_1 \in F$, 所以确定性有穷自动机 $\mathcal{M}_1$ 接受字符串 010。

3.2.4 扩展转移函数

扩展 δ 到字符串, 定义扩展转移函数 $\hat{\delta} : Q \times \Sigma^* \to Q$ 为

$$\hat{\delta}(q, w) = \begin{cases} q & w = \varepsilon \\ \delta(\hat{\delta}(q, x), a) & w = xa \end{cases}$$

其中 $q \in Q$, $a \in \Sigma$, $w, x \in \Sigma^*$, 并且我们有:

$$\hat{\delta}(q, xy) = \hat{\delta}(\hat{\delta}(q, x), y)$$

3.2.5 DFA 的语言

若 $M = (Q, \Sigma, \delta, q_0, F)$ 是一个确定性有穷自动机, 则 M 接受的语言为:

$$\mathcal{L}(\mathcal{M}) = \{w \in \Sigma^* | \hat{\delta}(q_0, w) \in F\}$$

3.3 非确定性有穷自动机

非确定性有穷自动机是一个 5 元组 $(Q, \Sigma, \delta, q_0, F)$, 其中:

1. Q 是一个有穷集合, 称为状态集
2. Σ 是一个有穷集合, 称为字母表
3. $\delta : Q \times \Sigma \to \mathcal{P}(Q)$ 是状态转移函数 $(\mathcal{P}(Q) = \{S | S \subseteq Q\})$
4. $q_0 \in Q$ 是起始状态
5. $F \subseteq Q$ 是接受状态集

一台非确定性有穷自动机有如上五个部分组成, 介绍如下:

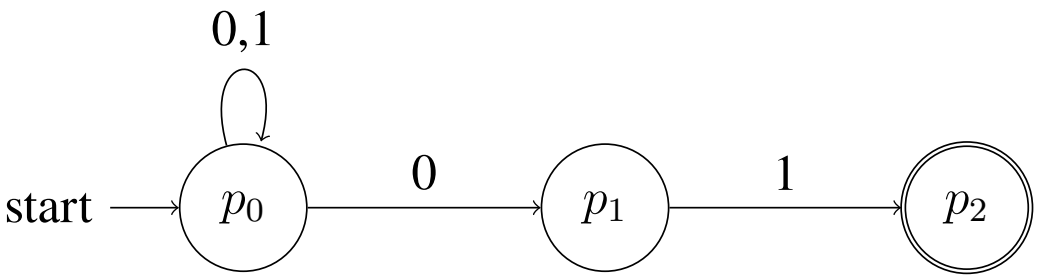
1. 它有一个状态集, 表示它有的全部状态
2. 它有一个输入字母表, 指明所有允许的输入符号
3. 它有一个根据一个输入字符从一个状态到零个或多个状态的规则
4. 它有一个起始状态, 表示处理所起始的状态
5. 它有一个接受状态集, 表示处理表达为接受的状态

3.3.1 图形表示

假如现在有个非确定性有穷自动机 $\mathcal{M}_2 = (Q, \Sigma, \delta, q_0, F)$

$$\begin{cases} Q = \{q_0, q_1, q_2\} \\ \Sigma = \{0, 1\} \\ \delta = \begin{cases} \delta(q_0, 0) = \{q_0, q_1\} \\ \delta(q_0, 1) = \{q_0\} \\ \delta(q_1, 0) = \varnothing \\ \delta(q_1, 1) = \{q_2\} \\ \delta(q_2, 0) = \varnothing \\ \delta(q_2, 1) = \varnothing \end{cases} \\ F = \{q_2\} \end{cases}$$

图形化表示就是:



3.3.2 用处

主要对于输入的字符串判定是否该非确定性有穷自动机所识别的语言, 对此表示接受及不接受。

而如上的确定性有穷自动机是识别以 01 结尾的串, 对于以 01 结尾 $x \in \Sigma^*$, 表示接受, 反之不接受。

非确定性有穷自动机可以用来定义语言也可用于识别语言。

3.3.3 运算

假如对于上面的确定性有穷自动机 $\mathcal{M}_2$, 输入字符串 010, 此时从初始状态 q_0 开始, 读入第一个字符 0, 因为 $\delta(q_0, 0) = \{q_0, q_1\}$, 此时有 2 个转移方向:

1. 转移状态至 q_0, 读入下一个字符 1, 因为 $\delta(q_0, 1) = \{q_0\}$, 所以转移状态至 q_0, 读入下一个字符 0, 因为 $\delta(q_0, 0) = \{q_0, q_1\}$, 此时有 2 个转移方向:
   (a). 转移状态至 q_0, 因为 $q_0 \notin F$, 所以不接受

(b). 转移状态至 q_1, 因为 $q_1 \notin F$, 所以不接受

2. 转移状态至 q_1, 读入下一个字符 1, 因为 $\delta(q_1, 1) = \{q_2\}$, 所以转移状态至 q_2, 读入下一个字符 0, 因为 $\delta(q_2, 0) = \varnothing$, 对于空集（就是没有规定这个状态对于这个输入的转移）, 我们认为此时等同于不接受

因为最后结果都是不接受, 所以我们认为该非确定性有穷自动机不接受该串。

如果有结果是接受, 才认为该非确定性有穷自动机接受该串。

3.3.4 扩展转移函数

扩展 δ 到字符串, 定义扩展转移函数 $\hat{\delta} : Q \times \Sigma^* \to \mathcal{P}(Q)$ 为:

$$\hat{\delta}(q, w) = \begin{cases} \{q\} & w = \varepsilon \\ \bigcup_{p \in \hat{\delta}(q,x)} \delta(p, a) & w = xa \end{cases}$$

其中 $q \in Q$, $a \in \Sigma$, $w, x \in \Sigma^*$

3.3.5 NFA 的语言

若 $\mathcal{M} = (Q, \Sigma, \delta, q_0, F)$ 是一个非确定性有穷自动机, 则 $\mathcal{M}$ 接受的语言为:

$$\mathcal{L}(\mathcal{M}) = \{w \in \Sigma^* | \hat{\delta}(q_0, w) \cap F \neq \varnothing\}$$

3.4 带空转移的非确定性有穷自动机

带空转移的非确定性有穷自动机是一个 5 元组 $(Q, \Sigma, \delta, q_0, F)$, 其中:

1. Q 是一个有穷集合, 称为状态集
2. Σ 是一个有穷集合, 称为字母表
3. $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \to \mathcal{P}(Q)$ 是状态转移函数 $(\mathcal{P}(Q) = \{S | S \subseteq Q\})$
4. $q_0 \in Q$ 是起始状态
5. $F \subseteq Q$ 是接受状态集

一台非确定性有穷自动机有如上五个部分组成, 介绍如下:

1. 它有一个状态集, 表示它有的全部状态
2. 它有一个输入字母表, 指明所有允许的输入符号
3. 它有一个根据零个或一个输入字符从一个状态到零个或多个状态的规则
4. 它有一个起始状态, 表示处理所起始的状态
5. 它有一个接受状态集, 表示处理表达为接受的状态

3.4.1 状态的 ε-闭包

状态 q 的 ε-闭包 (ε-*Closure*), 记为 $E_{CLOSE}(q)$, 表示从 q 经过 $\varepsilon\varepsilon\cdots\varepsilon$ 序列可达的全部状态集合, 定义为:

$$\begin{cases} q \in E_{CLOSE}(q) \\ \forall p \in E_{CLOSE}(q), r \in \delta(p, \varepsilon) \to r \in E_{CLOSE}(q) \end{cases}$$

状态集 Q 的 ε-闭包为:

$$E_{CLOSE}(Q) = \bigcup_{q \in Q} E_{CLOSE}(q)$$

3.4.2 图形表示

与 NFA 不同的是, 此时可以用字母表中没有的空转移字符 ε 来做转移字符, 其余与 NFA 相同

3.4.3 用处

与 NFA 相同

3.4.4 运算

遇到 ε 时, 就不消耗输入的字符就进行状态转移, 其他与 NFA 相同

3.4.5 扩展转移函数

扩展 δ 到字符串, 定义扩展转移函数 $\hat{\delta} : Q \times \Sigma^* \to 2^Q$ 为:

$$\hat{\delta}(q, w) = \begin{cases} E_{CLOSE}(q) & w = \varepsilon \\ E_{CLOSE}(\bigcup_{p \in \hat{\delta}(q,x)} \delta(p, a)) & w = xa \end{cases}$$

其中 $q \in Q$, $a \in \Sigma$, $w, x \in \Sigma^*$

3.4.6 ε-NFA 的语言

若 $\mathcal{M} = (Q, \Sigma, \delta, q_0, F)$ 是一个带空转移的非确定性有穷自动机, 则 $\mathcal{M}$ 接受的语言为:

$$\mathcal{L}(\mathcal{M}) = \{w \in \Sigma^* | \hat{\delta}(q_0, w) \cap F \neq \varnothing\}$$

如果语言 $\mathcal{L}$ 被 ε-NFA 接受, 当且仅当 $\mathcal{L}$ 被 DFA 接受。

3.5 状态转移表

对于转移函数

$$\delta = \begin{cases} \delta(q_0, 0) = q_1 \\ \delta(q_0, 1) = q_0 \\ \delta(q_1, 0) = q_1 \\ \delta(q_1, 1) = q_0 \end{cases}$$

我们可以将其用表来表达出来:

	0	1
q_0	q_1	q_0
q_1	q_1	q_0

对于转移函数

$$\delta = \begin{cases} \delta(q_0, 0) = \{q_0, q_1\} \\ \delta(q_0, 1) = \{q_0\} \\ \delta(q_1, 0) = \varnothing \\ \delta(q_1, 1) = \{q_2\} \\ \delta(q_2, 0) = \varnothing \\ \delta(q_2, 1) = \varnothing \end{cases}$$

我们也可以将其用表来表达出来:

	0	1
q_0	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\varnothing$	$\{q_2\}$
q_2	$\varnothing$	$\varnothing$

3.6 正则语言

如果语言 $\mathcal{L}$ 是某个确定性有穷自动机 $\mathcal{M}$ 的语言, 即 $\mathcal{L} = \mathcal{L}(\mathcal{M})$, 则称 $\mathcal{L}$ 为正则语言。

1. $\varnothing, \{\varepsilon\}$ 都是正则语言
2. 若 Σ 是字母表, 则有 Σ^*, Σ^n 都是 Σ 上的正则语言

3.7 NFA 与 DFA 识别能力等价

NFA 与 DFA 在语言识别能力上是等价的, 所有的 NFA 都有与之对应的 DFA, 将 NFA 转换至 DFA 的方法称为**子集构造法**。

3.8 子集构造法

对于上面 NFA 的例子, 非确定性有穷自动机 $\mathcal{M}_2$, 假设其转换后的 DFA 为 $\mathcal{N}$, 我们可以针对状态转移表下手:

	0	1
q_0	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\varnothing$	$\{q_2\}$
q_2	$\varnothing$	$\varnothing$

我们可以将状态这一列用集合的方式来表示, 首先 q_0 变成 $\{q_0\}$, 0/1 对应的转移保留

	0	1
$\{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$

此时出现了一个新状态 $\{q_0, q_1\}$, 将 M_2 中 q_0 和 q_1 中 0 对应的转移状态取并集, 1 也同

	0	1
$\{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$

此时出现了一个新状态 $\{q_0, q_2\}$, 将 M_2 中 q_0 和 q_2 中 0 对应的转移状态取并集, 1 也同

	0	1
$\{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$
$\{q_0, q_2\}$	$\{q_0, q_1\}$	$\{q_0\}$

此时我们用 q_0 代替 $\{q_0\}$, 用 q_1 代替 $\{q_0, q_1\}$, 用 q_2 代替 $\{q_0, q_2\}$, 整个表就变成

	0	1
q_0	q_1	q_0
q_1	q_1	q_2
q_2	q_1	q_0

这就是其对应的确定性有穷自动机 $\mathcal{N}$ 的状态转移表, 因为 $\mathcal{M}_2$ 的接受状态集 F 是 $\{q_2\}$, 而上面只有 $\{q_0, q_2\}$ 包含 q_2, 即新构建出的 $\mathcal{N}$ 的状态转移表中的 q_2, 所以这就是 $\mathcal{N}$ 的接受状态集。

假设 $\mathcal{N} = (Q, \Sigma, \delta, q_0, F)$, 我们有:

$$\begin{cases} Q = \{q_0, q_1, q_2\} \\ \Sigma = \{0, 1\} \\ \delta = \begin{cases} \delta(q_0, 0) = q_1 \\ \delta(q_0, 1) = q_0 \\ \delta(q_1, 0) = q_1 \\ \delta(q_1, 1) = q_2 \\ \delta(q_2, 0) = q_1 \\ \delta(q_2, 1) = q_0 \end{cases} \\ F = \{q_2\} \end{cases}$$

对于带空转移的非确定性有穷自动机 $\mathcal{E} = (Q_E, \Sigma, \delta_E, q_E, F_E)$, 构造确定性有穷自动机 $\mathcal{D} = (Q_D, \Sigma, \delta_D, q_D, F_D)$, 我们有

1. $Q_D = \mathcal{P}(Q_E)$ 或 $Q_D = \{S \subseteq Q_E | S = E_{CLOSE}(S)\}$
2. $q_D = E_{CLOSE}(q_E)$
3. $F_D = \{S | S \in Q_D, S \cap F_E \neq \varnothing\}$
4. $\forall S \in Q_D, \forall a \in \Sigma, \delta_D(S, a) = E_{CLOSE}\left(\bigcup_{p \in S} \delta_E(p, a)\right)$

   那么 $\mathcal{L}(\mathcal{D}) = \mathcal{L}(\mathcal{E})$

3.9 时间复杂度

我们这里并不会利用**图灵机**给出**时间复杂度**的形式化定义, 这对于大部分读者来说是难以理解的。这里会像一般的程序课程一样, 以编程语言某一段执行为标准代替**确定性图灵机**经过的步数来进行计算。

3.9.1 Θ

Θ 定义的是一段程序的执行步数的上下界 (等于), 比如对于程序:

```
for (int i = 0; i < m; ++i)
    for (int j = 0; j < n; ++j)
        [Subroutine A]
```

我们以 **Subroutine A** 为标准记步, 易知对于输入 **m,n** 有时间复杂度为 $\Theta(mn)$

3.9.2 O,o,Ω,ω

O 定义的是一段程序的执行步数的上界 (小于等于)。

o 定义的是一段程序的执行步数的上界 (小于)。

Ω 定义的是一段程序的执行步数的下界 (大于等于)。

ω 定义的是一段程序的执行步数的下界 (大于)。

3.10 P 与 NP 与 NPC

P 类问题是可以在**多项式时间**内解决的问题, 而 **NP** 类问题则是可以在**多项式时间**内验证的问题, 易知当一个问题属于 **P** 类时, 这个问题同属 **NP** 类问题。

著名的 **P=NP** 问题则是当一个问题属于 **NP** 类时, 这个问题是否同属 **P** 类问题。这个问题自 **1971 年**被提出后, 已经成为**理论计算机科学**领域最深奥、最令人费解的开放问题之一。

而有一类问题叫做 **NPC** 类问题, 或者说叫做 **NP 完全**类问题, 这表示:

1. 它是一个 **NP** 类问题
2. 其他 **NP** 类问题都可在**多项式时间**内归约成它

也就是如果任何 **NP 完全**类问题可以在**多项式时间**内解决, 那么所有 **NP** 类问题都有一个**多项式时间**算法。

第 4 章 算法与数据结构

内容提要

- 算法是为了求解某一类问题的良性定义的一系列过程
- 可以很简单的用递推来计算出一个数列的前缀和

算法是为了求解某一类问题的良性定义的一系列**过程**, 而**数据结构**则是为了更好的存储数据和辅助**算法**求解的一系列**结构**。本章就基础**算法**和**数据结构**进行讲解。

4.1 快速幂与快速乘

4.1.1 快速幂

根据数学常识, 每一个正整数都可以唯一表示为若干指数不重复的 2 的次幂之和。也就是说, 如果 b 在二进制表示下有 k 位, 其中第 i $(0 \le i < k)$ 位的数字是 c_i, 那么:

$$b = c_{k-1} * 2^{k-1} + c_{k-2} * 2^{k-2} + \cdots + c_{k-1} * 2^0$$

于是有:

$$a^b = a^{c_{k-1}*2^{k-1}} * a^{c_{k-2}*2^{k-2}} * \cdots * a^{c_0*2^0}$$

因为 $k = \lceil \log_2 b + 1 \rceil$, 所以上式乘积项的数量不多于 $\lceil \log_2 b + 1 \rceil$ 个。又因为:

$$a^{2^i} = \left(a^{2^{i-1}}\right)^2$$

所以我们很容易通过 k 次递推求出每个乘积项, 当 $c_i = 1$ 时, 把该乘积项累积到答案中。整个算法的时间复杂度为 $O(log_2 b)$。

```
int pow(int a, int b, int p) {
    int ans = 1;
    for (; b; b >>= 1) {
        if (b & 1)
            ans = (long long) ans * a % p;
        a = (long long) a * a % p;
    }
    return ans;
}
```

4.1.2 快速乘

类似**快速幂**, 把整数 b 用二进制表示, 即:

$$b = c_{k-1} * 2^{k-1} + c_{k-2} * 2^{k-2} + \cdots + c_{k-1} * 2^0$$

那么:

$$a * b = c_{k-1} * a * 2^{k-1} + c_{k-2} * a * 2^{k-2} + \cdots + c_0 * a * 2^0$$

又因为:

$$a * 2^i = (a * 2^{i-1}) * 2$$

所以我们很容易通过 k 次递推求出每个乘积项, 当 $c_i = 1$ 时, 把该乘积项累加到答案中即可。时间复杂度为 $O(log_2 b)$。

```
long long mul(long long a, long long b, long long p) {
    long long ans = 0;
    for (; b; b >>= 1) {
        if (b & 1)
            ans = (ans + a) % p;
        a = (a + a) % p;
    }
    return ans;
}
```

4.2 前缀和与差分

4.2.1 前缀和

对于一个给定的数列 A, 它的前缀和数列 S 的定义是:

$$S_i = \sum_{j=1}^{i} A_j$$

你可以很简单的用递推来计算出一个数列的前缀和:

$$S_i = \begin{cases} A_1 & i = 1 \\ S_{i-1} + A_i & i > 1 \end{cases}$$

一个部分和即数列 A 某个下标区间内的数的和, 可表示为前缀和相减的形式:

$$sum(l,r) = \sum_{j=l}^{r} A_j = S_r - S_{l-1}$$

```
int S[SIZE];

void init(int* A, int n) {
    S[0] = A[0];
    for (int i = 1; i < n; ++i)
        S[i] = S[i - 1] + A[i];
}

int add(int l, int r) {
    return S[r] - S[l - 1];
}
```

在二维数组（矩阵）中, 也可类似地求出二维前缀和, 进一步计算出二维部分和。

4.2.2 差分

对于一个给定的数列 A, 它的差分数列 B 定义为:

$$B_i = \begin{cases} A_1 & i = 1 \\ A_i - A_{i-1} & i > 1 \end{cases}$$

把数列 A 的区间 [l,r] 加 d, 其差分数列 B 的变化为:

1. $B_l = B_l + d$
2. $B_{r+1} = B_{r+1} - d$

这样可以使得我们把原数列上的“区间操作”转化为差分数列上的“单点操作”进行计算, 降低求解难度。

4.3 栈与队列

4.3.1 栈

栈 (stack) 是一种**后进先出**的数据结构, **栈**只有一端能够进出元素, 我们通常称这一端为**栈顶**, 另一端为**栈底**。

比如进栈顺序为 a, b, c, d, 弹出的顺序为 d, c, b, a。

4.3.2 队列

队列 (queue) 是一种是**先进先出**的数据结构, **队列**只允许在后端进行插入操作, 在前端进行删除操作。

元素多次入队、出队后, 用于实现队列结构的数组的开头部分空间就会被严重浪费, 所以我们经常将其优化成**循环队列**, 也就是把队列看作一个首位相接的环, 只要队列中的元素个数在任意时刻都不超过环长, 那么随着入队和出队操作的进行, 存储元素的那一段位置就像沿着环不停的移动, 重复利用着曾被占用过的空间。

比如进队顺序为 a, b, c, d, 弹出的顺序为 a, b, c, d。

4.4 链表

对于常见的**数组**, 它支持**随机访问**, 不支持在任意位置插入或删除元素。于是引入了一种数据结构叫**链表**, 它只支持**顺序访问**, 但支持在任意位置插入或删除元素。

我们可以用一个**结构体**来表示**链表**的节点, 并且用 **prev** 和 **next** 两个指针指向前后相邻的两个节点, 另外用两个节点 **head** 和 **tail** 代表链表头尾:

```
typedef struct Node {
    int value;
    struct Node *prev, *next;
} Node;

Node *head, *tail;
```

初始化链表:

```
head = (Node *) malloc(sizeof(Node));
tail = (Node *) malloc(sizeof(Node));
head->next = tail;
tail->prev = head;
```

在链表插入元素:

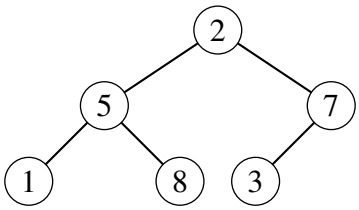
```
Node *new = (Node *) malloc(sizeof(Node));
new->value = val;
p->next->prev = new;
new->next = p->next;
p->next = new;
new->prev = p;
```

在链表删除元素:

```
p->prev->next = p->next;
p->next->prev = p->prev;
free(p);
```

4.5 二叉树

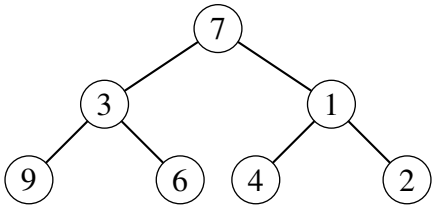
二**叉树**是一种树, 其每个节点有 $0 \sim 2$ 个节点, 例如:



通常分支被称作**左子树**或**右子树**。二**叉树**的分支具有左右次序, 不能随意颠倒。

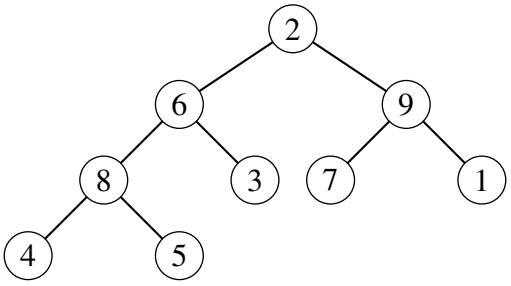
4.5.1 满二叉树

一个深度为 k 且有 $2^{k+1}-1$ 个结点的二**叉树**称为**完美二叉树**, 例如:



4.5.2 完全二叉树

完全二叉树从根结点到倒数第二层满足**满二叉树**, 最后一层可以不完全填充, 其叶子结点都靠左对齐, 例如:

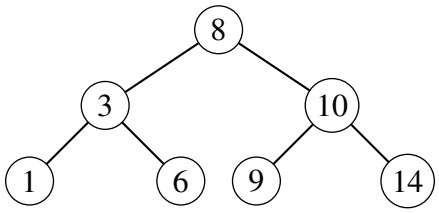


4.6 二叉搜索树

二叉搜索树又叫**二叉查找树**、**二叉排序树**, 是一种满足以下性质的**二叉树**:

1. 若任意节点的**左子树**不空, 则**左子树**上所有节点的值均小于它的**根节点**的值
2. 若任意节点的**右子树**不空, 则**右子树**上所有节点的值均大于它的**根节点**的值
3. 任意节点的**左**、**右子树**也分别为**二叉搜索树**

例如:



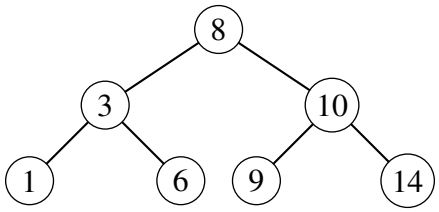
二叉搜索树相比于其他数据结构的优势在于**查找**和**插入**的时间复杂度较低, 为 $O(\log n)$。

4.7 深度优先搜索

深度优先搜索步骤:

1. 访问顶点 v
2. 依次从 v 的未被访问的**邻接点**出发, 对图进行**深度优先搜索**, 直至图中和 v 有路径相通的顶点都被访问
3. 若此时图中尚有顶点未被访问, 则从一个未被访问的顶点出发, 重新进行**深度优先搜索**, 直到图中所有顶点均被访问过为止

例如:



深度优先搜索的顺序是: 8->3->1->6->10->9->14

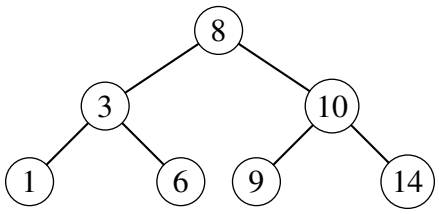
4.8 广度优先搜索

广度优先搜索步骤:

1. 首先将根节点放入队列中
2. 从队列中取出第一个节点, 并检验它是否为目标。如果找到目标, 则结束搜寻并回传结果。否则将它所有尚未检验过的直接子节点加入**队列**中

3. 若**队列**为空，表示整张图都检查过了——亦即图中没有欲搜寻的目标。结束搜寻并回传“找不到目标”

   例如:



广度优先搜索的顺序是: 8->3->10->1->6->9->14

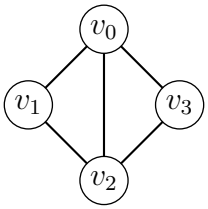
4.9 邻接表与邻接矩阵

图的存储结构主要分两种，一种是**邻接矩阵**，一种是**邻接表**。这里仅举例**无向图**的，**有向图**只需添加**权值**即可。请参照本节内容自学**有向图**的**邻接表**与**邻接矩阵**表示方法作为练习。

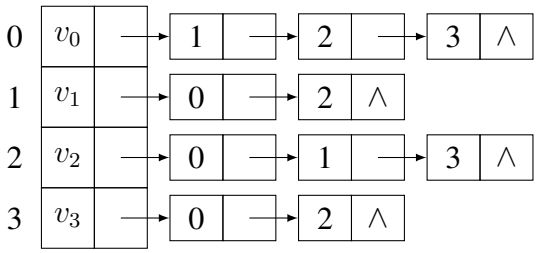
4.9.1 邻接表

有一种**数组**与**链表**相结合的存储方法称为**邻接表**，给图中的各个结点独自建立一个**链表**，并存在**数组**中，用该**链表**存储该结点所有的**邻接点**。

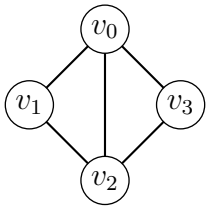
例如对于图:



我们有:



4.9.2 邻接矩阵



可表示为:

$$\begin{array}{cc} & \begin{array}{cccc} v_0 & v_1 & v_2 & v_3 \end{array} \\ \begin{array}{c} v_0 \\ v_1 \\ v_2 \\ v_3 \end{array} & \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \end{bmatrix} \end{array}$$

4.10 单源最短路径

给定带权有向图 $G = (V, E)$, 和权重函数 $w : E \to R$
图中一条路径 $p =< v_0, v_1, \cdots, v_n >$ 则有

$$w(p) = \sum_{k=1}^{n} w(v_{k-1}, v_k)$$

定义从节点 u 到节点 v 的最短路径权重 $\delta(u, v)$ 如下:

$$\delta(u, v) = \begin{cases} min\{w(p) : u \overset{p}{\rightsquigarrow} v\} & \text{If there is a path from u to v} \\ \infty \end{cases}$$

从节点 u 到节点 v 的最短路径则定义为任何一条权重为 $w(p) = \delta(u, v)$ 的从 u 到 v 的路径.

4.10.1 性质

对于所有的 $v \in V$, $v.d$ 表示从源节点 s 到 v 的**最短路径估计**

4.10.1.1 性质一

给定带权有向图 $G = (V, E)$, 和权重函数 $w : E \to R$

设 $p =< v_0, v_1, \cdots, v_n >$ 为从节点 v_0 到节点 v_n 的一条**最短路径**, 并且对于任意的 i 和 j, 都有 $0 \le i \le j \le n$

设 $p_{ij} =< v_i, v_{i+1}, \cdots, v_j >$ 为路径 p 中从节点 v_i 到节点 v_j 的子路径。那么 p_{ij} 是从节点 v_i 到节点 v_j 的一条**最短路径**。

4.10.1.2 性质二 (三角不等式性质)

$$\forall (u, v) \in E, \delta(s, v) \le \delta(s, u) + \delta(u, v)$$

4.10.1.3 性质三 (上界性质)

对于所有节点 $v \in V$, 我们总是有 $v.d \ge \delta(s, v)$, 一旦 $v.d = \delta(s, v)$, 其值将不再发生变化。

4.10.1.4 性质四 (非路径性质)

若 $s \not\rightsquigarrow v$, 则总是有 $\delta(s, v) = \infty$

4.10.1.5 性质五

对于某些节点 $u, v \in V$, 若 $s \rightsquigarrow u \to v$ 是图 G 的一条**最短路径**, 并且在对边 (u, v) 进行松弛前的任意时间有 $u.d = \delta(s, u)$, 则在之后的所有时间, 有 $v.d = \delta(s, v)$

4.10.1.6 性质六

若 $p =< v_0, v_1, \cdots, v_n >$ 是从源节点 $s = v_0$ 到节点 v_n 的一条**最短路径**, 并且我们对 p 中是边进行松弛的次序为:

$$(v_0, v_1), (v_1, v_2), \cdots, (v_{n-1}, v_n)$$

则 $v_n.d = \delta(s, v_n)$

该性质的成立与任何其他的**松弛操作**无关, 即使这些**松弛操作**是与对 p 上的边所进行的**松弛操作**穿插进行的

4.10.1.7 性质七

对于所有的节点 $v \in V$, 一旦 $v.d = \delta(s, v)$, 则**前驱子图**是一颗根节点为 s 的**最短路径树**。

4.10.2 求法

常见的求法有比如 **Dijkstra 算法**、**Bellman-Ford 算法**和 **SPFA 算法**。

4.11 Dijkstra 算法

Dijkstra 算法流程如下:

设 $dist[i]$ 为从起点 1 到节点 i 的最短路径长度, (x, y, z) 表示从 x 到 y, 权值为 z 的边。

1. 初始化 $dist[1] = 0$, 其余节点的 $dist$ 值为 ∞
2. 找出一个未被标记的、$dist[x]$ 最小的节点 x, 然后标记 x
3. 扫描节点 x 的所有出边 (x, y, z), 若 $dist[y] > dist[x] + z$, 则使用 $dist[x] + z$ 更新 $dist[y]$
4. 重复 $2 \sim 3$, 直到所有节点被标记

Dijkstra 算法基于贪心思想, 适用于所有边的长度都是非负数的图。

代码如下:

```
memset(d, 0x3f, sizeof(d));
memset(v, 0, sizeof(v));
d[1] = 0;
for (int i = 1; i < n; ++i) {
    int x = 0;
    for (int j = 1; i <= n; ++j)
        if (!v[j] && (x == 0 || d[j] < d[x]))
            x = j;
        v[x] = 1;
        for (int y = 1; y <= n; ++y)
            d[y] = min(d[y], d[x] + a[x][y]);
}
```

4.12 Bellman-Ford 算法

Bellman-Ford 算法流程如下:

设 $dist[i]$ 为从起点 1 到节点 i 的最短路径长度, (x, y, z) 表示从 x 到 y, 权值为 z 的边。

1. 扫描所有边 (x, y, z), 若 $dist[y] > dist[x] + z$, 则使用 $dist[x] + z$ 更新 $dist[y]$
2. 重复上述步骤, 直到没有更新操作发生

而 **SPFA 算法**实际上是**队列优化的 Bellman-Ford 算法**。

SPFA 算法流程如下:

1. 建立一个队列, 最初队列中只含有起点 1
2. 取出队头节点 x, 扫描它的所有出边 (x, y, z), 若 $dist[y] > dist[x] + z$, 则使用 $dist[x] + z$ 更新 $dist[y]$。同时, 若 y 不在队列中, 则把 y 入队
3. 重复上述步骤, 直到队列为空

4.13 哈密顿回路

$G=(V,E)$ 是一个图, 若 G 中一条路径通过且仅通过每一个顶点一次, 称这条路径为**哈密顿路径**。若 G 中一个回路通过且仅通过每一个顶点一次, 称这个环为**哈密顿回路**。若一个图存在**哈密顿回路**, 就称为**哈密顿图**。

4.14 旅行商问题

给定一系列城市和每对城市之间的距离, 求解访问每一座城市一次并回到起始城市的最短回路。旅行商问题是一个 NPC 问题。

4.15 矩阵

数学上, 一个 $m \times n$ 的矩阵是一个由 m 行 n 列元素排列成的矩形阵列。矩阵里的元素可以是数字、符号或数学式。

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & \dots & a_{1j} & \dots & a_{1n} \\ a_{21} & a_{22} & a_{23} & \dots & a_{2j} & \dots & a_{2n} \\ a_{31} & a_{32} & a_{33} & \dots & a_{3j} & \dots & a_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ a_{i1} & a_{i2} & a_{i3} & \dots & a_{ij} & \dots & a_{in} \\ \vdots & \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & a_{m3} & \dots & a_{mj} & \dots & a_{mn} \end{bmatrix}$$

4.15.1 矩阵乘法

矩阵相乘只有在第一个矩阵的列数和第二个矩阵的行数相同时才有意义。

设矩阵 A 为 $P \times M$ 的, B 为 $M \times Q$ 的, 设矩阵 C 为 A 与 B 的乘积, 则有:

$$C_{ij} = \sum_{k=1}^{M} A_{ik} B_{kj}$$

4.16 矩阵加速递推

我们假设存在矩阵 M 使得 $\begin{pmatrix} F_n \\ F_{n+1} \end{pmatrix} = M \begin{pmatrix} F_{n-1} \\ F_n \end{pmatrix}$

设 $M = \begin{pmatrix} A & C \\ B & D \end{pmatrix}$, 就有 $\begin{pmatrix} F_n \\ F_{n+1} \end{pmatrix} = M \begin{pmatrix} AF_{n-1} + CF_n \\ BF_{n-1} + DF_n \end{pmatrix}$

易知 $\begin{cases} A = 0 \\ C = B = D = 1 \end{cases}$, 即 $M = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}$

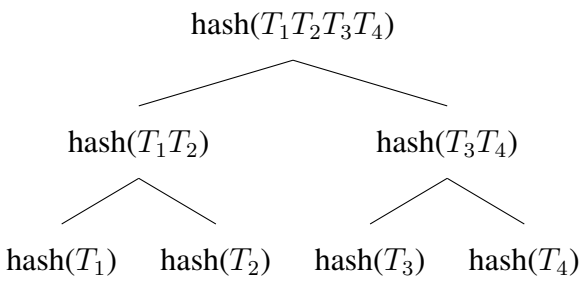
于是就有:

$$\begin{pmatrix} F_n \\ F_{n+1} \end{pmatrix} = M^n \begin{pmatrix} F_0 \\ F_1 \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}^n \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

于是, 我们就可以利用**矩阵**矩阵加速递推求出斐波纳契数列第 n 项的值。

4.17 哈希树

对于序列 T_1, T_2, T_3, T_4, 我们可以通过二分的形式计算 **hash** 值, 例如:



4.18 二进制状态压缩

二进制状态压缩是将一个长度为 m 的 bool 数组用一个 m 位二进制整数表示并存储的方法。具体操作如下:

操作	运算
取出整数 n 在二进制表示下的第 k 位	$(n >> k)\&1$
取出整数 n 在二进制表示下的第 $0 \sim k-1$ 位	$n\&((1 << k) - 1)$
将整数 n 在二进制表示下的第 k 位取反	$n\ xor\ (1 << k)$
将整数 n 在二进制表示下的第 k 位赋 1	$n\|(1 << k)$
将整数 n 在二进制表示下的第 k 位赋 0	$n\&(\sim (1 << k))$

4.19 动态规划

动态规划是通过把原问题分解为相对简单的子问题的方式求解复杂问题的方法。

它的三要素为**状态**、**阶段**、**决策**, 而应用的三前提则为:

1. 无后效性: 求解不受后续求解影响
2. 最优子结构性: 子问题最优解合并全局最优解
3. 子问题重叠性: 同子问题反复求解

动态规划算法把相同的计算过程作用于各阶段的同类子问题。这个过程被称为**状态转移方程**。

常见的问题有:

LIS 问题	问题描述	**最长上升子序列** 给定一个长度为 N 的数列 A 求数值单调递增的子序列的长度最长是多少
	状态表示	$F[i]$ 表示以 $A[i]$ 结尾的**最长上升子序列**的长度
	阶段划分	子序列的结尾位置
	转移方程	$F[i] = \max\limits_{0 \le j < i, A[j] < A[i]} \{F[j] + 1\}$
	边界	F[0]=0
	目标	$\max\limits_{1 \le i \le N} \{F[i]\}$
LCS 问题	问题描述	**最长公共子序列** 给定两个长度分别为 N 和 M 的字符串 A 和 B 求即是 A 的子序列又是 B 的子序列的字符串长度最长是多少
	状态表示	$F[i,j]$ 表示前缀子串 $A[1 \sim i]$ 与 $B[1 \sim j]$ 的**最长公共子序列**的长度
	阶段划分	已处理的前缀长度
	转移方程	$F[i,j] = max \begin{cases} F[i-1,j] \\ F[i,j-1] \\ F[i-1,j-1]+1 \quad if\ A[i]=B[j] \end{cases}$
	边界	$F[i,0] = F[0,j] = 0$
	目标	$F[N,M]$
数字三角形	问题描述	给定一个共有 N 行的三角矩阵 A, 其中第 i 行有 i 列 从左上角除法, 每次可以向下方或右下方走一步, 最终到达底部 求把经过的所有位置上的数加起来, 和最大是多少
	状态表示	$F[i,j]$ 表示从左上角走到第 i 行第 j 列, 和最大是多少
	阶段划分	路径的结尾位置
	转移方程	$F[i,j] = A[i,j] + max \begin{cases} F[i-1,j] \\ F[i-1,j-1]+1 \quad if\ j>1 \end{cases}$
	边界	$F[1,1] = A[1,1]$
	目标	$\max\limits_{1 \le j \le N} \{F[N,j]\}$

4.20 背包

背包是**动态规划**一类重要而特殊的模型, 这里举例 **0/1 背包**。

0/1 背包	问题描述	给定 N 个物品, 其中第 i 个物品的体积为 V_i, 价值为 W_i 有一容积为 M 的背包, 要求选择一些物品放进背包 使得背包总体积不超过 M 的前提下, 物品的价值总和最大
	状态表示	$F[i,j]$ 表示从前 i 个物品中选取出总体积为 i 的物品放入背包 背包的最大价值和
	阶段划分	已经处理的物品数
	转移方程	$F[i,j] = max \begin{cases} F[i-1,j] & \text{unselect item i} \\ F[i-1,j-V_i]+W_i & \text{select item i} \end{cases}$
	边界	$F[m,n] = \begin{cases} 0 & m=n=0 \\ \infty & \end{cases}$
	目标	$\underset{0\le j\le M}{max}\{F[N,j]\}$

4.21 并查集

并查集 (Disjoint-Set) 是一种可以动态维护若干个不重叠的集合, 并支持合并与查询的数据结构。详细的说, **并查集**包含如下两个基本操作:

1. Get: 查询一个元素属于哪一个集合
2. Merge: 把两个集合合并成一个大集合

在**并查集**中, 我们采用 **"代表元"** 法, 即为每个集合选取一个固定的元素, 作为整个集合的 **"代表"**。

其次, 我们要定义归属关系的表示方法。我们使用一个树形结构存储着每个集合, 树上的每个节点都是一个元素, 树根是集合的代表元素。整个**并查集**实际上是一个森林, 我们维护一个数组 fa 来记录这个森林, 用 $fa[x]$ 保存 x 的父节点。特别的, 令树根的 fa 值为它自己。这样以来, 在合并两个集合时, 只需连接两个树根。

4.21.1 路径压缩

在每次执行 Get 操作的同时, 把访问过的每个节点都直接指向树根。

第 5 章 密码学

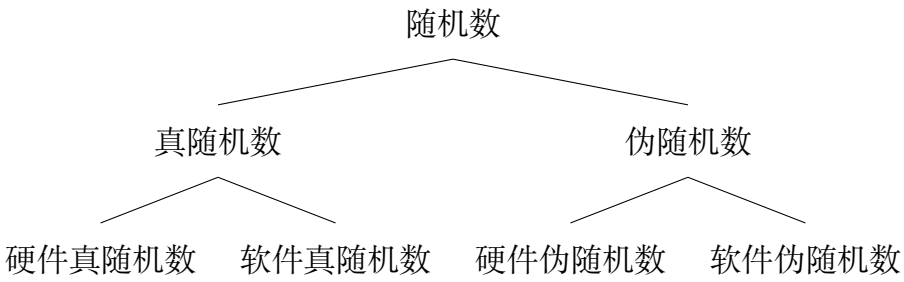
内容提要

- 一次一密是无条件安全的
- CSPRNG 是一种不可预测的 PRNG
- 对伪随机数生成器一个一般要求是，它必须拥有良好的统计属性

在互联网世界中，信息的保密尤其重要，特别是帐号密码等信息在互联网上传输，危机重重。在本章将介绍一些基本的**密码学**知识。

5.1 随机数生成器

通常来说，**随机数生成器**有两种，一是**真随机数生成器**，二是**伪随机数生成器**。而**伪随机数生成器**有一个特例，那就是**加密安全的伪随机数生成器**。通常来说，**真随机数生成器**一般是**硬件**实现的，而**伪随机数生成器**则一般是**软件**实现的。



5.1.1 真随机数生成器

真随机数生成器 (TRNG) 一般是通过**硬件**实现的，如**电子元件噪音**、**核裂变**等。其特点就是具有**不可预测性**，有的哲学家宣称“世界上不存在真正的随机”，至于世界上是否有真正的随机确实是一个耐人寻味的问题，但是为了现实应用以及统计学的需要，我们需要暂时忽略这种哲学范畴的问题。

5.1.2 伪随机数生成器

伪随机数生成器 (PRNG) 一般是通过**软件**实现的，比如从一个初始的**种子**开始通过递归的执行以下计算得到：

$$s_0 = seed$$
$$s_{i+1} = f(s_i)$$

此表达式的一个推广形式是 $s_{i+1} = f(s_i, s_{i-1}, \cdots, s_{i-t})$ 所示的生成器, 其中 t 是一个固定的数。最常见的例子就是**线性同余生成器**:

$$s_0 = seed$$
$$s_{i+1} = (a * s_i + b) \bmod m$$

对**伪随机数生成器**一个一般要求是, 它必须拥有良好的统计属性。

5.1.3 加密安全的伪随机数生成器

加密安全的伪随机数生成器 (CSPRNG) 通常是指具有不可预测的**伪随机数生成器**。

5.2 哈希函数

哈希函数能够将**任意长度**的输入值转变成**固定长度**的值输出, 该值称为**散列值**, 输出值通常为**字母**与**数字**组合。常见的**哈希函数**有 **MD5**、**SHA256** 等。

假设采用的**哈希函数**为 $hash$, 如果对于 $a! = b$ 有:

$$hash(a) = hash(b)$$

则我们称呼其为**哈希冲突**。

通常来说, 一个合格的**哈希函数**具有以下特点:

1. 不可逆性
2. 产生固定长度的输出值
3. 计算时间不应过长
4. 能接受长度非常长的输入值
5. 输入值不变, 输出的值也是唯一不变的
6. 求冲突在计算上不可行
7. 即使修改了输入值的一点, 也会使得输出值发生巨大的变化
8. 产生的映射应当保持均匀

5.3 古典密码

古典密码有很多, 多为**替换密码**, 其典型的例子就是**凯撒密码**, 其只是简单的把**字母表**中的字母向后移动相同的位数。当达到**字母表**尾部时, 将从开头开始循环。

5.4 序列密码

序列密码是一种**对称加密算法**, 。其单独加密每个位。依照原理, 我们而可以将**序列密码**分为**同步序列密码**和**异步序列密码**。其中, **同步序列密码**的密码仅取决于密钥, 而**同步序列密码**则取决于**密钥**和**密文**。

明文、密文和密钥序列都是由单独的位组成的, 即 $x_i, y_i, s_i \in \{0, 1\}$

加密: $y_i = e_{s_i}(x_i)$

解密: $x_i = d_{s_i}(y_i)$

5.5 一次一密

一个**序列密码**称为**一次一密**, 必须满足以下条件:

1. 通过真随机数生成器得到密钥序列
2. 只有合法的通信方才知道密码序列
3. 每个密钥只使用一次

一次一密是无条件安全的。

5.6 对称加密

通常我们把加解密所使用的密钥相同的称为**对称加密**:

数据加密过程: 在对称加密算法中, **数据发送方**将**明文 (原始数据)** 和**加密密钥**一起经过特殊**加密处理**, 生成复杂的**加密密文**进行发送。

数据解密过程: **数据接收方**收到密文后, 若想读取原数据, 则需要使用**加密使用的密钥**及相同算法的**逆算法**对加密的密文进行解密, 才能使其恢复成**可读明文**。

常见的比如: **DES 算法**、**AES 算法**

5.7 非对称加密

通常我们把加解密所使用的密钥分为**公钥**和**私钥**的称为**非对称加密**。

1. 如果使用**公钥**对**数据**进行**加密**, 只有用对应的**私钥**才能进行**解密**
2. 如果使用**私钥**对**数据**进行**加密**, 只有用对应的**公钥**才能进行**解密**

常见的比如: **RSA 算法**

5.8 数字签名

数字签名简单来说就是通过提供**可鉴别**的数字信息**验证**自身**身份**的一种方式。一套**数字签名**通常定义两种**互补**的运算, 一个用于**签名**, 另一个用于**验证**。分别由**发送者**持有能够**代表自己身份**的**私钥 (私钥不可泄露)**, 由**接受者**持有与私钥对应的**公钥**, 能够在**接受**到来自发送者信息时用于**验证**其身份。

常见的比如: **ECDSA 算法**

第 6 章 计算机组成

内容提要

- 冯 · 诺依曼体系的核心思想主要是指数字计算机的数制采用二进制以及计算机应该按照程序顺序执行
- 哈佛结构是一种将程序指令存储和数据存储分开的存储器结构

6.1 冯 · 诺依曼计算机

1949 年, 著名的计算机科学家**冯 · 诺依曼 (John von Neumann)** 创建了现今计算机多采用的**冯 · 诺依曼体系**, 而采用这种体系的计算机都被称为**冯 · 诺依曼计算机**。

冯 · 诺依曼体系的核心思想主要是指数字计算机的数制采用二进制以及计算机应该按照程序顺序执行。根据这种思想实现的计算机主要要具有以下的功能:

1. 把需要的程序和数据送至计算机中
2. 必须具有长期记忆程序、数据、中间结果及最终运算结果的能力
3. 够完成各种算术、逻辑运算和数据传送等数据加工处理的能力
4. 能够根据需要控制程序走向, 并能根据指令控制机器的各部件协调操作
5. 能够按照要求将处理结果输出给用户

要实现上述五个功能, 计算机必须具备五大基本组成部件:

1. 输入数据和程序的输入设备
2. 记忆程序和数据的存储器
3. 完成数据加工处理的运算器
4. 控制程序执行的控制器
5. 输出处理结果的输出设备

6.2 存储器概念

存储器的主要功能是存放**程序**和**数据**。**程序**是计算机操作的依据, **数据**是计算机操作的对象。不管是**程序**还是**数据**, 在存储器中都是用二进制数的形式表示的, 统称为信息。向**存储器**存入和从**存储器**取出数据, 都称为**访问存储器**。

计算机**存储器**是由可以存放和读取数值的一系列单元所组成的, 每个**存储单元**都有一个编号, 称为**地址**。向**存储器**中存**数据**或者从**存储器**取**数据**, 都要按给定的**地址**来寻找所选的**存储单元**。存放在**存储器**中的信息, 事实上可以表示任何东西, 文字、数值甚至计算机指令都可以同样容易的存放到**存储器**中。

由于计算机仅使用 0 和 1 两个二进制数字, 所以使用**位 (bit)** 作为数字计算的最小信息单位。当 CPU 向**存储器**送入或从**存储器**取出信息时, 不能存放单个位, 而使用 **字节 (Byte)** 和**字 (Word)** 等较大的信息单位来工作。一个字节由 8 个二进制信息组成, 而一个字则表示计算机一次能处理的一组二进制数, 至少由一个以上的字节组成。通常把组成一个字的二进制位数叫做字长。例如微型机的字长可以少至 8 位, 多至 32 位, 甚至达到 64 位。

存储器中所有**存储单位**的总数称为**存储器**的**存储容量**, 通常用单位 **KB (Kilobyte, 千字节)**、**MB (Megabyte, 兆字节)**、**GB (gigabyte, 千兆字节)** 表示, 如 64KB、128MB、256GB。度量**存储器**容量的各级单位之间的关系为:

1. 1KB = 1000Byte
2. 1MB = 1000KB
3. 1GB = 1000MB

存储容量越大, 计算机所能记忆存储的信息就越多。

存储器是计算机中存储信息的部件。按照存储器在计算机中的作用又可分为 **主存储器**、**寄存器**、**闪存**、**高速缓冲存储器**、**辅助存储器**等几种类型, 它们均可完成数据的存取工作, 但性能及其在计算机中的作用相差很大。

6.3 寄存器

寄存器 (Register) 的主要功能是存储二进制代码, 用于存放参与运算的数据和运算结果。

首先, 寄存器的工作频率要大幅度的和 CPU 运算器一致 (节拍一致), 每一次存取数据所需速度都要非常非常的少, 花费零点几纳秒的时间就完成数据的读取, 所以寄存器的成本十分高昂。

根据功能不同寄存器也分为两种, **基本寄存器**和**移位寄存器**, 基本寄存器只能**并行送入数据**, 也只能并行输出数据。而移位寄存器可以在移位脉冲作用下依次逐位右移或左移, 既可以并行输入、并行输出, 也可以串行输入、串行输出, 还可以并行输入、串行输出, 或串行输入、并行输出。

寄存器最起码具备以下 4 种功能:

1. 清除: 将寄存器里的原有数据清除
2. 接收: 在接收脉冲作用下, 将外输入数据存入寄存器中
3. 存储: 在没有新的写入脉冲来之前, 寄存器能保存原有数据不变
4. 输出: 在输出脉冲作用下, 才通过电路输出数据

6.4 随机存储器

随机存储器 (Random Access Memory) 是主存的一种, 用于存放供 CPU 使用的绝大部分程序和数据, 既可向指定单元存入信息又可从指定单元读出信息, 必须带电存储, 断电数据丢失, 所以随机存储器是**易失性存储器**。

而所谓的随机存储，主要是指一段数据被读取或写入的时候，花费的时间与这段数据所存储的位置无关。相对的，**写入顺序访问存储器**被读取或写入的时候，花费的时间就与这段数据所存储的位置有关了。而随机存储器主要分两大类，**动态随机存储器 (DRAM)** 和**静态随机存储器 (SRAM)**

6.4.1 动态随机存储器

动态随机存储器 (Dynamic Random Access Memory) 利用栅极电容存储电荷的原理保存信息，所以会存在电荷泄漏的现象，而为了解决这种现象，就需要时隔几毫秒对存储单元进行读取和写入，刷新的时间必须小于栅极电容自然保持信息的时间，这个过程被成为 DRAM 的刷新。它的功耗较 SRAM 来说更小，而因为刷新的存在，DRAM 的外围电路会比较复杂，刷新也使得 DRAM 的存储速度较 SRAM 来说更慢，而因为其成本低廉，设计简单，成为现今大容量 RAM 的主流。

6.4.2 静态随机存储器

静态随机存储器 (Static Random Access Memory) 是在静态触发器的基础上附加门控管而构成的，因此，它是靠触发器的自保功能存储数据的。它存放的数据能在不断电的前提下长期保存，状态稳定，也不需要进行刷新，所以外围电路设计简单，功耗较 DRAM 来说也更大，速度也较 DRAM 来说更快，但以目前的情况来说，人类无法以低廉的成本造出 SRAM，所以这东西经常被用来作为 CPU 的 Cache。

6.5 只读存储器

只读存储器 (Read Only Memory) 也是主存的一种，正如起名，是只能读取且以非破坏性读取的存储器，信息一旦写入后就固定下来，即便切断电源后数据仍然存在不会丢失，所以只读存储器是**非易失性存储器**。**只读存储器**又有多种不同的版本，比如 PROM、EPROM、EEPROM、CDROM 等。

6.5.1 可编程只读存储器

可编程只读存储器 (Programmable Read Only Memory) 可以使用 PROM 编程器将数据写入 (又被称为编程)，在将数据写入后，就只能专用那些数据，并且不能被再次编程。所以又被称为**一次可编程只读存储器 (One Time Progarmming ROM)**，简称 **“OTP-ROM”**。

6.5.2 可擦除可编程只读存储器

可擦除可编程只读存储器 (Erasable Programmable Read Only Memory) 指可以重复擦除数据并且可以重复写入且可以重复读取的只读存储器。通常由一组浮栅晶体管构成，被一个提

供比电子电路中常用电压更高电压的电子器件分别编程。进行编程后只能使用强紫外线照射来擦除, 在以前常被用作存储 BIOS。

6.5.3 带电可擦除可编程只读存储器

带电可擦除可编程只读存储器 (Electrically Erasable Programmable Read Only Memory) 指可以通过高于普通电压的作用来擦除和重编程只读存储器, 也就是可以通过电擦除和写入的 EPROM。EEPROM 以 Byte 为最小修改单位, 不必将数据全部洗掉才能写入。

6.6 闪存

Flash 是一种存储芯片, 通过特定的程序可以修改里面的数据, 它是一种非易失存储介质, 在写入前需要擦除。原指由**桀冈富士雄 (Fujio Masuoka)** 发明的 **NOR Flash**, 但在 1986 年, 桀冈富士雄又发明了 **NAND Flash**, 这时 Flash 泛指 **NOR Flash** 和 **NAND Flash**。

6.6.1 NOR Flash

NOR Flash 是**桀冈富士雄 (Fujio Masuoka)** 在 1980 年发明的, 当时桀冈富士雄申请了一个叫做 **Simultaneously Erasable EEPRO** 的专利。**NOR Flash** 的一个 bit 可以从 1 变成 0, 而从 0 变成 1 就需要擦除整块。**NOR Flash** 的容量较 **NAND Flash** 来说比较小, 读取速度较 **NAND Flash** 来说很快, 而写入速度就差强人意。擦除速度也较 **NAND Flash** 来说很慢。通常来说它的可擦除次数在 $10^4 \sim 10^5$ 内, 可靠性也较 **NAND Flash** 来说更高, 当然价格也高。**NOR Flash** 支持随机访问, 这也是的它具有支持**芯片内执行 (XIP, eXecute In Place)** 的特性, 可以像普通 ROM 一样执行程序。这点使得它成为了 BIOS 等开机就要执行代码的绝佳载体。

现在几乎所有的 BIOS 和一些机顶盒上都是使用 **NOR Flash**, 它的大小主要在 $1MB \sim 32MB$ 之间, 价格高昂。

6.6.2 NAND Flash

NAND Flash 也是**桀冈富士雄 (Fujio Masuoka)** 在 1986 年发明的, **NAND Flash** 的一个 bit 要使 1 变成 0 或 0 变成 1 都需要进行擦除, 容量较 **NOR Flash** 来说更大, 读取速度较 **NOR Flash** 来说慢一些, 而写入速度就快一点。擦除速度也较 **NOR Flash** 来说很快。通常来说它的可擦除次数在 $10^5 \sim 10^6$ 内, 可靠性也较 **NAND Flash** 来说低一点, 当然价格也低。**NAND Flash** 是按块方式读取, 也就不支持**芯片内执行 (XIP, eXecute In Place)** 了。我们电脑里的固态硬盘, 大多都是采用 **NAND Flash** 的, 它广泛应用在各种存储卡、U 盘、SSD、eMMC 等等大容量设备中。

NAND Flash 的颗粒, 根据单元内存存储比特个数的不同, 也被分为 **SLC (Single Level Cell)**、**MLC (Multi Level Cell)**、**TLC (Triple Level Cell)**、**QLC (Quadra Level Cell)** 四类, 其中

在一个存储单元中, SLC 可以存储 1 个比特, MLC 可以存储 2 个比特, TLC 可以存储 3 个比特, QLC 可以存储 4 个比特, NAND Flash 的单个存储单元存储的比特位越多, 读写性能就会越差, 寿命也越短, 对主控芯片的要求就越高, 但是成本就会更低。

主要影响 **NAND Flash** 寿命的因素, 就是每块的擦写次数。而 **NAND Flash** 也会随着使用而渐渐产生坏块, 如何平衡各块的擦写和为可能的坏块寻找替换呢? 通常需要一个特殊的软件层次, 实现坏块的管理、擦写均衡、ECC、垃圾回收等功能, 这一软件层次被称为 **FTL (Flash Translation Layer)**。

6.7 哈佛结构

哈佛结构 (Harvard Architecture) 是一种将程序指令存储和数据存储分开的存储器结构。哈佛结构是一种并行体系结构, 它的主要特点是将程序和数据存储在不同的存储空间中, 即程序存储器和数据存储器是两个独立的存储器, 每个存储器独立编址、独立访问。在哈佛架构中 CPU 首先到程序指令存储池中读取程序指令内容, 解码后得到数据地址, 再到相应的数据存储器中读取数据, 并进行下一步操作 (通常是执行), 程序指令和数据存储分开, 数据和指令的存储可以同时进行, 也使指令和数据有不同的数据宽度。

6.8 高速缓冲存储器

在许多复杂的计算机中, 存在一个或多个比寄存器慢但比主存快的高速 RAM 缓冲存储器 (简称**高速缓存**) ——Cache。通常使用 SRAM。它是位于 CPU 和主存储器之间、规模较小但速度很快的存储器, 很好地解决了 CPU 和主存之间的速度匹配。

第 7 章 组合数学

内容提要

- 若完成一件事的方法有 n 类，其中第 i 类方法包括 a_i 种不同的方法，且这些方法互不重合，则完成这件事共有 $\sum_{k=1}^{n} a_k$ 种不同的方法
- 若完成一件事的方法需要 n 个步骤，其中第 i 个步骤有 a_i 种不同的完成方法，且这些方法互不干扰，则完成这件事共有 $\prod_{k=1}^{n} a_k$ 种不同的方法

7.1 加法原理

若完成一件事的方法有 n 类，其中第 i 类方法包括 a_i 种不同的方法，且这些方法互不重合，则完成这件事共有 $\sum_{k=1}^{n} a_k$ 种不同的方法。

7.2 乘法原理

若完成一件事的方法需要 n 个步骤，其中第 i 个步骤有 a_i 种不同的完成方法，且这些方法互不干扰，则完成这件事共有 $\prod_{k=1}^{n} a_k$ 种不同的方法。

7.3 排列数

从 n 个不同元素中依次取出 m 个元素排成一列，产生的不同排列的数量为:

$$A_n^m = \frac{n!}{(n-m)!} = n * (n-1) * \cdots * (n-m+1)$$

7.4 组合数

从 n 个不同元素中取出 m 个组成一个集合，产生的不同集合数量为:

$$C_n^m = \frac{n!}{m!(n-m)!} = \frac{n * (n-1) * \cdots * (n-m+1)}{m * (m-1) * \cdots * 2 * 1}$$

7.4.1 性质

1. $C_n^m = C_n^{n-m}$
2. $C_n^m = C_{n-1}^m + C_{n-1}^{m-1}$

3. $\sum_{k=0}^{n} C_n^k = 2^n$

7.5 二项式定理

$$(a+b)^n = \sum_{k=0}^{n} C_n^k a^k b^{n-k}$$

7.6 卢卡斯定理

若 p 是**素数**, 则对于任意整数 $1 \le m \le n$, 有:

$$C_n^m \equiv C_{n \bmod p}^{m \bmod p} * C_{\frac{n}{p}}^{\frac{m}{p}} \pmod p$$

7.7 卡塔兰数列

给定 n 个 0 和 n 个 1, 它们按照某种顺序排列成长度为 $2n$ 的序列, 满足任意前缀 0 的个数都不小于 1 的个数的序列的数量为:

$$Cat_n = \frac{C_{2n}^n}{n+1}$$

7.8 容斥原理

设 $A_1, A_2, \cdots, A_n$ 为有限集合, 则:

$$\left|\bigcup_{i=1}^{n} A_i\right| = \sum_{\emptyset \ne J \subseteq \{1,2,\ldots,n\}} (-1)^{|J|-1} \left|\bigcap_{j \in J} A_j\right|$$

7.9 莫比乌斯函数

记正整数 N 按照**算数基本定理**分解质因数为 $N = p_1^{c_1} p_2^{c_2} \cdots p_m^{c_m}$, 定义函数:

$$\mu(N) = \begin{cases} 0 & \exists i \in [1,m], c_i > 1 \\ 1 & m \equiv 0 \pmod 2, \forall i \in [1,m], c_i = 1 \\ -1 & m \equiv 1 \pmod 2, \forall i \in [1,m], c_i = 1 \end{cases}$$

称 $\mu(N)$ 为**莫比乌斯函数**。

第 8 章 杂项

内容提要

- 证明了“若程序终止，则获得正确的答案”，可以说证明了程序的部分正确性
- 若 $(p \wedge condition)\{S\}p$ 为真，则 p 是循环不变量

8.1 程序正确性证明

对于一个程序，如果我们要形式化验证这段程序的正确性，可以使用**霍尔逻辑**。

霍尔逻辑是一套由英国计算机科学家**霍尔 (C.A.R.Hoare)** 开发的形式系统，主要是用于推理验证程序的正确性。

8.1.1 程序正确性证明

若对于每个可能的输入，程序都产生正确的输出，则说这个程序是正确的。

证明分为两个部分：

1. 若程序终止，则获得正确的答案
2. 程序总是终止

证明了 1，可以说证明了程序的部分正确性。

8.1.2 断言

1. 初始断言：输入值必须具有的性质
2. 终结断言：程序做了正确的事后，输出值必须具有的性质

8.1.3 霍尔三元组

若对于一个程序或程序段 S 的输入值来说，初始断言 p 为真时，就有对于 S 的输出值来说，终结断言 q 为真，则说 S 是相对于 p 和 q 部分正确的。计为：

$$p\{S\}q$$

8.1.4 合成规则

通过把一个程序分解成一系列子程序，通过证明这些子程序为正确的，来证明这个程序是正确的。

如把程序 S 分解为子程序 S_1 和 S_2, 则可写作

$$S = S_1; S_2$$

假设 p 和 r 分别为程序 S 的初始断言和终结断言, 而 q 为子程序 S_1 的**终结断言**、子程序 S_2 的**初始断言**。可叙述成:

$$\frac{\begin{array}{c} p\{S_1\}q \\ q\{S_2\}r \end{array}}{\therefore p\{S\}r}$$

8.1.4.1 定义

对于程序 $S = S_1; S_2; \cdots; S_{n-1}; S_n$, 可叙述成:

$$\frac{\begin{array}{c} p_1\{S_1\}p_2 \\ p_2\{S_2\}p_3 \\ \vdots \\ p_{n-1}\{S_n\}p_n \end{array}}{\therefore p_1\{S_1; S_2; \cdots; S_{n-1}; S_n\}p_n}$$

8.1.5 验证程序段

8.1.5.1 程序段一

```
if condition then
    S1
else
    S2
```

$$\frac{\begin{array}{c} (p \wedge condition)\{S_1\}q \\ (p \wedge \neg condition)\{S_2\}q \end{array}}{\therefore p\{if\ condition\ then\ S1\ else\ S2\}q}$$

8.1.5.2 程序段二

```
if condition then
    S
```

$$\frac{\begin{array}{c}(p \wedge condition)\{S\}q \\ (p \wedge \neg condition)\{S\} \rightarrow q\end{array}}{\therefore p\{if\ condition\ then\ S\}q}$$

8.1.6 循环不变量

对于程序:

```
1     if condition then
2         S
```

S 反复执行直到 $condition$ 为假为止, 所以必须选择一个每次执行 S 都保持为真的断言, 而这个断言就是**循环不变量**。

对于如上的程序, 验证如下:

$$\frac{(p \wedge condition)\{S\}p}{\therefore p\{while\ condition\ S\}(p \wedge \neg condition)}$$

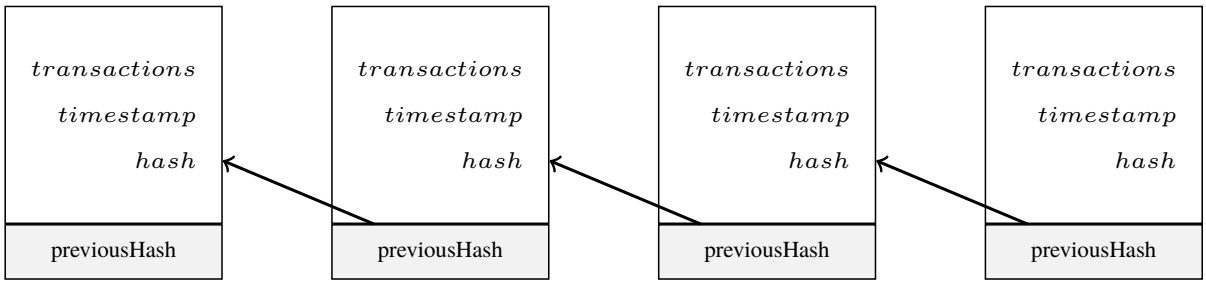
若 $(p \wedge condition)\{S\}p$ 为真, 则 p 是**循环不变量**。

8.2 区块链

区块链自**比特币**提出以来, 被应用到各种领域, 比如**商品溯源**等。本文就**虚拟货币**为目标, 讲解一个**区块链**所实现的**虚拟货币**。

8.2.1 数据结构

一个**区块链**是通过区块内指向**前区块 Hash** 来串联在一起的, 形如:



而一个区块的 **hash** 是通过**时间戳**、**交易表 Hash**、**随机数**来计算出来的, 而**交易表 Hash** 则是通过**哈希树**计算出来的。整个**区块链**具备不可篡改等性质。

8.2.2 分布式账本与去中心化

我们将整个**账本**放在每一个节点的上, 每个节点都有能力去验证**账本**。

而**账本**通过 **p2p** 在节点与节点之间传输, 没有真实意义上的**中心节点**。特点是两个节点在相互转移**虚拟货币**时, 无须**中心节点**的参与。

8.2.3 共识机制

为了选取一个区块的**打包者**, 通常采用被称为 **"共识机制"** 的东西来进行确定。最常见的就是**工作量证明机制**, 比如说计算 **hash** 值, 优先算出 **hash** 前 n 位为 0 的节点成为**打包者**。

8.2.4 激励机制

通常**打包者**会获得一定的 "成果", 比如说系统会给**打包者**一定的**虚拟货币**作为奖励。

8.2.5 签名

在发起**交易**的时候, 通常采用例如 **ECDSA** 这样的**签名算法**对**交易**进行签名, 以证明交易的正确性。